

# CurveMole User Manual

Preview 0.42.0

Sebastiano Romi

European Laboratory for Non-Linear Spectroscopy (LENS)
University of Florence (UNIFI)

romi@lens.unifi.it

Updated for CurveMole 0.42.0

GPL-3.0-or-later

## Contents

**1. About this manual** **6**
1.1 Intended audience . . . 6
1.2 Version policy for the manual . . . 6
1.3 Terminology and notation . . . 6
1.4 Scientific responsibility . . . 7

**2. Core concepts** **8**
2.1 Project . . . 8
2.2 Series and curves . . . 8
2.3 Model, component, and parameter . . . 8
2.4 Active, selected, and visible curves . . . 8
2.5 Fit plan and result . . . 9

**3. Installation and first launch** **10**
3.1 Reference requirements . . . 10
3.2 Desktop packages . . . 10
3.3 Linux AppImage . . . 10
3.4 Verifying SHA-256 checksums . . . 11
3.5 Installing the Python package . . . 11
3.6 Running from a source checkout . . . 12
3.7 Confirming the installed version . . . 12
3.8 Version badge and updates . . . 12

**4. A complete first fit** **13**
4.1 Import the example . . . 13
4.2 Add a background . . . 13
4.3 Add a Gaussian peak graphically . . . 13
4.4 Adjust the model before fitting . . . 14
4.5 Mask an interval if needed . . . 14
4.6 Run the fit . . . 14
4.7 Save and export . . . 14

**5. Main window and controls** **15**
5.1 Series and curves tree . . . 15
5.2 Plot workspace . . . 16
5.3 Navigation and axes . . . 17
5.4 Model and parameters dock . . . 17
5.5 Quick actions . . . 18
5.6 Dockable tools . . . 19
5.7 Themes . . . 19
5.8 Drag and drop . . . 19

**6. Importing data** **20**
6.1 Supported text formats . . . 20
6.2 Comments, blank lines, and encoding . . . 20
6.3 Column mapping . . . 22
6.4 Batch import . . . 23
Calibrated WinSpec SPE files . . . 23
Automatic folder import . . . 23
6.5 Invalid numeric cells . . . 23
6.6 Uncertainties and weights . . . 24

**7. Curves, masks, and transformations** **25**
7.1 Immutable originals . . . 25
7.2 Masking points and intervals . . . 25
7.3 Mask targets . . . 25
7.4 Undo and mask history . . . 25
7.5 Data Calculator . . . 26
Advanced column formulas . . . 26
7.6 Curve-to-curve calculations . . . 28
7.7 Restoring original data . . . 28
7.8 Worksheet . . . 28
7.9 Editing individual data points . . . 28

**8. Building models** **30**
8.1 Component composition . . . 30
8.2 Built-in peak parameterization . . . 31
Gaussian . . . 31
Lorentzian . . . 31
Voigt . . . 31
Pseudo-Voigt . . . 31
8.3 Built-in functions commonly used as backgrounds . . . 32
Constant . . . 32
Linear . . . 32
Polynomial . . . 32
Cubic spline . . . 32
8.4 Quick Add Function . . . 33
8.5 Adding a peak with the pointer . . . 33
8.6 Initializing any function from manually selected points . . . 33
Drawing a cubic-spline background . . . 34
8.7 Graphical editing after placement . . . 34
8.8 Automatic peak suggestions . . . 34
8.9 Duplicating, disabling, deleting, and reordering . . . 35
8.10 Copying a fit to other curves . . . 35

**9. Parameters, constraints, and links** **36**
9.1 Parameter table . . . 36
Choose a source spectrum . . . 36
Build an expression from several parameters . . . 37
Choose a quick relationship . . . 37
9.2 Parameter states . . . 38
9.3 Parameter paths . . . 38
9.4 Link evaluation . . . 38
9.5 Safe expression language . . . 39

**10. Fitting** **40**
10.1 Pre-fit checklist . . . 40
10.2 Fit modes . . . 40
Single / independent . . . 40
Sequential . . . 40
Global simultaneous . . . 41
10.3 Per-spectrum weights . . . 41
10.4 Local least squares . . . 41
10.4.1 Advanced algorithm options and Reset default . . . 42
10.5 Robust losses . . . 43
10.6 Differential Evolution initial search . . . 43
10.7 Cancellation and failed fits . . . 43
10.8 Common causes of failure . . . 44

**11. Results, statistics, and diagnostics** **45**
11.1 Visual results . . . 45

11.2 Reported global statistics . . . 45
11.3 Per-curve statistics . . . 45
11.4 Covariance and correlation . . . 45
11.5 Residual diagnostics . . . 46

**12. Explicit uncertainty analyses** **47**
12.1 Parametric Monte Carlo . . . 50
12.2 Residual bootstrap . . . 50
12.3 Block bootstrap . . . 50
12.4 Profile likelihood . . . 50
12.5 Reproducibility and failed replicates . . . 51

**13. Projects, saving, and recovery** **52**
13.1 The `.fitproj` format . . . 52
13.2 Atomic saves . . . 52
13.3 Project locks and read-only mode . . . 52
13.4 Normal save and portable copy . . . 52
13.5 Autosave recovery . . . 52
13.6 Unsaved changes . . . 53

**14. Exporting an analysis bundle** **54**
14.1 Starting an export . . . 54
14.2 Default fit-results file . . . 54
14.3 Optional outputs . . . 54
14.4 Safe overwrite policy . . . 55
14.5 Wide data tables . . . 55
14.5.1 Exporting optimizer and loss settings . . . 55
14.6 Tidy data and JSON . . . 55
14.7 Reports and reusable files . . . 55
14.8 Exporting one numeric file per spectrum . . . 56
14.9 Exporting function parameters . . . 56

**15. Command-line interface** **57**
15.1 General form . . . 57
15.2 Open the GUI . . . 57
15.3 List available functions . . . 57
15.4 Fit one file . . . 57
15.5 Fit a series . . . 57
15.6 Inspect and validate . . . 58
15.7 Export an existing project . . . 58
15.8 Exit codes . . . 58

**16. Reproducible YAML workflows** **59**
16.1 Running a workflow . . . 59
Run an existing automation from the desktop . . . 59
16.2 Complete example . . . 59
16.3 Custom formulas in YAML . . . 60
16.4 Trusted plugins . . . 60

**17. Python API** **61**
17.1 Stable package-level objects . . . 61
17.2 Minimal fit . . . 61
17.3 Bounds and fixed values . . . 61
17.4 Cross-curve links . . . 62
17.5 Global fit . . . 62
17.6 API compatibility during Preview . . . 62

**18. Function Builder and plugins** **63**
18.1 Formula Builder . . . 63
18.2 Custom function persistence . . . 63

18.3 Local plugin structure . . . 64
18.4 Trust boundary . . . 64
18.5 Plugin Manager . . . 64
18.6 Creating extensions beyond model functions . . . 66
18.7 Community plugin submissions and validation . . . 66
18.8 Acquisition processors and interactive plugin panels . . . 67

**19. Troubleshooting** **68**
19.1 The application does not start on Linux . . . 68
19.2 Quick Guide, manual, updates, or issue links do not open . . . 68
19.3 Windows SmartScreen or macOS Gatekeeper warns . . . 68
19.4 Import produces one column . . . 68
19.5 Imported points disappear from fitting . . . 68
19.6 A peak appears at the wrong height after placement . . . 68
19.7 A spline looks unstable . . . 69
19.8 Differential Evolution searches too narrow an interval . . . 69
19.9 The fit is underdetermined or covariance is unavailable . . . 69
19.10 A parameter remains at a bound . . . 69
19.11 A sequential fit pauses . . . 69
19.12 A project opens read-only . . . 69
19.13 Export refuses to overwrite . . . 69
19.14 A resampling analysis has many failed replicates . . . 70
19.15 An update is detected but cannot be installed automatically . . . 70
19.16 Reporting a problem . . . 70

**20. Reproducible research, privacy, and citation** **71**
20.1 Recommended research record . . . 71
20.1.1 Laboratory notebook . . . 71
20.2 Local processing and privacy . . . 71
20.3 Citation . . . 72
20.4 Author and contact . . . 72
20.5 License . . . 72

**Appendix A. Keyboard shortcuts** **73**

**Appendix B. Built-in identifiers** **74**

**Appendix C. File extensions** **75**

**Appendix D. Preview 0.42.0 limitations** **76**

**Appendix E. Building this manual** **77**

CurveMole is a desktop-first, scriptable application for fitting one-dimensional scientific curves. This manual describes the behavior of CurveMole 0.42.0 Preview. The Markdown file is the authoritative source. The LaTeX source and PDF edition are generated automatically from it and must carry the same version number.

> **Preview status.** CurveMole 0.42.0 is suitable for evaluation and controlled testing. It has not yet completed the scientific validation planned for version 1.0.0. Inspect the residuals, parameter correlations, constraints, and exported results before using a fit in research.

Useful links:

- CurveMole repository
- Latest release and desktop downloads
- Issue tracker
- Version DOI and citation
- Quick Start

# 1. About this manual

## 1.1 Intended audience

This manual is for researchers and technical users who need to fit spectra or other x-y data while retaining control over data preparation, model construction, constraints, uncertainty, and export. Typical data include:

- infrared and Raman spectra;
- powder diffraction patterns;
- kinetic traces;
- chromatographic profiles;
- temperature, pressure, or field-dependent signals;
- any other one-dimensional curve that can be represented by numeric x and y columns.

No programming knowledge is required for the graphical workflow. Separate chapters cover the command-line interface, YAML workflows, Python API, formula builder, and plugins.

## 1.2 Version policy for the manual

The version in the title and in the line at the top of this file identifies the CurveMole release whose behavior is documented. It is not an independent document version. For example, a manual marked 0.42.0 describes CurveMole 0.42.0.

The release automation performs four checks:

1. the application version and both manual version declarations must agree;
2. all local links in the Markdown source must resolve;
3. Pandoc must generate a standalone LaTeX document without warnings;
4. XeLaTeX must compile the generated source into a valid PDF.

The release assets use versioned names so that a downloaded manual can always be associated with the correct executable.

## 1.3 Terminology and notation

Menu commands are written as **File > Import data**. Buttons and field names are shown in **bold**. Commands, filenames, identifiers, and parameter paths use `monospace` text.

CurveMole uses the following residual convention in displays and exports:

$$r_i = y_i - f(x_i).$$

The optimizer minimizes the opposite sign, $f(x_i) - y_i$, after applying uncertainty and spectrum scaling. The sign has no effect on a least-squares minimum but matters when reading exported residuals.

### 1.4 Scientific responsibility

CurveMole can optimize a mathematically defined model, but it cannot decide whether that model is physically appropriate. A small residual or a high descriptive $R^2$ does not by itself establish uniqueness, chemical meaning, or predictive validity. In particular:

- do not add components solely to improve a scalar fit statistic;
- use bounds only when they have a defensible meaning;
- inspect correlations and active bounds;
- check whether residuals contain structure;
- report the exact CurveMole version, model, constraints, weighting, and uncertainty method used.

## 2. Core concepts

CurveMole keeps the scientific engine independent from the graphical interface. The desktop application, command-line interface, YAML workflows, and Python API all operate on the same domain objects.

### 2.1 Project

A project is the complete analysis workspace. It can contain data, models, masks, transformations, fit results, custom formulas, history, and interface state. Projects are saved as `.fitproj` files.

### 2.2 Series and curves

A **series** groups related curves. A **curve** contains one x array and one y array, with optional `sigma_x`, `sigma_y`, or point weights. Importing multiple y columns from a common file produces multiple curves in one imported series.

Original numeric arrays are stored as immutable 64-bit floating-point values. Data Calculator operations create reversible transformations instead of overwriting the original arrays.

### 2.3 Model, component, and parameter

Each curve has its own model. A model is an ordered sequence of components such as a background and several peaks. Every component has:

- a function, for example `gaussian` or `linear`;
- a user-facing name;
- a composition operator;
- a set of parameters;
- an enabled or disabled state;
- optional metadata, such as polynomial order or spline x nodes.

A parameter has a current value, optional lower and upper bounds, a fixed state, and an optional expression link. Standard errors and confidence limits are populated after a successful fit when they are available.

### 2.4 Active, selected, and visible curves

These states are deliberately distinct:

- **Active curve:** the single curve whose model and handles are being edited.
- **Selected curves:** one or more curves chosen for multi-curve operations.
- **Visible curves:** curves drawn in Overlay or Waterfall display modes.

Activating a curve does not automatically select every visible curve. Before a fit, mask transfer, or multi-curve calculation, check the target shown in the relevant dialog.

### 2.5 Fit plan and result

A fit plan specifies the curves, fit mode, solver settings, per-spectrum weights, and whether numerical contributions should be equalized. A fit result records the optimized parameters, statistics, residual arrays, covariance, correlation, warnings, timestamp, and solver status.

# 3. Installation and first launch

## 3.1 Reference requirements

The current reference configuration is:

| Item | Minimum or supported target |
|---|---|
| Processor | x86-64 or Apple Silicon on macOS; 4 CPU cores recommended |
| Memory | 8 GB RAM |
| Storage | SSD recommended |
| Display | 1920 x 1080 recommended |
| Windows | Windows 10 22H2 legacy compatibility or Windows 11, 64-bit |
| macOS | macOS 13 Ventura or newer, Intel or Apple Silicon |
| Linux | Ubuntu 22.04 LTS, Debian 12, or a comparable newer distribution |
| Python | Python 3.12 or newer for Python installations |

A dedicated GPU is not required. Large multi-curve fits and resampling analyses may require more memory and time than the reference configuration.

## 3.2 Desktop packages

Open the latest release and select the package for your system.

| Platform | Release asset | Normal launch method |
|---|---|---|
| Linux x86-64 | `CurveMole-VERSION-linux-x86_64.AppImage` | Mark executable and run |
| Windows x86-64 | `CurveMole-VERSION-windows-x86_64-setup.exe` | Run the per-user installer |
| Windows portable | `CurveMole-VERSION-windows-x86_64-portable.zip` | Extract and run `CurveMole.exe` |
| macOS Apple Silicon | `CurveMole-VERSION-macos-arm64.dmg` | Drag CurveMole to Applications |
| macOS Intel | `CurveMole-VERSION-macos-x86_64.dmg` | Drag CurveMole to Applications |

Windows and macOS packages are currently unsigned. The operating system may show a warning on first launch. Download only from the official GitHub release page and verify the checksum if the application will be used for research.

## 3.3 Linux AppImage

In a terminal opened in the download directory:

```
chmod +x CurveMole-0.42.0-linux-x86_64.AppImage
./CurveMole-0.42.0-linux-x86_64.AppImage
```

If the system cannot mount AppImages through FUSE, use extraction mode:

```
APPIMAGE_EXTRACT_AND_RUN=1 ./CurveMole-0.42.0-linux-x86_64.AppImage
```

Some minimal Linux installations may need graphical runtime libraries supplied by the distribution. On Debian or Ubuntu, the commonly required packages are:

```
sudo apt install libegl1 libgl1 libxkbcommon-x11-0 libxcb-cursor0
```

Choose **Tools > Integrate CurveMole with the desktop** to copy a new AppImage to a stable per-user location, add CurveMole to the application menu, install its icon, and register `.fitproj` projects. No administrator privileges are required. If a working manual `curvemole.desktop` launcher already exists, CurveMole backs it up once and adopts its command rather than creating a duplicate application entry.

The Windows installer similarly creates a Start-menu entry, offers an optional desktop shortcut, and registers `.fitproj`. On macOS the application appears in Applications and Launchpad. On every platform, a registered project can be opened by double-clicking it or choosing **Open with CurveMole**.

### 3.4 Verifying SHA-256 checksums

Every release includes `SHA256SUMS.txt`. On Linux:

```
sha256sum -c SHA256SUMS.txt --ignore-missing
```

On macOS:

```
shasum -a 256 CurveMole-0.42.0-macos-arm64.dmg
```

On Windows PowerShell:

```
Get-FileHash .\CurveMole-0.42.0-windows-x86_64-setup.exe -Algorithm SHA256
```

Compare the reported value with the corresponding line in `SHA256SUMS.txt`.

### 3.5 Installing the Python package

Use an isolated environment. From a downloaded wheel:

```
python3 -m venv ~/.venv/curvemole
source ~/.venv/curvemole/bin/activate
python -m pip install ./curvemole-0.42.0-py3-none-any.whl
curvemole gui
```

On Windows PowerShell, activation is:

```
py -m venv $env:USERPROFILE\.venv\curvemole
& $env:USERPROFILE\.venv\curvemole\Scripts\Activate.ps1
python -m pip install .\curvemole-0.42.0-py3-none-any.whl
curvemole gui
```

### 3.6 Running from a source checkout

Clone the repository and install its locked development environment:

```
git clone https://github.com/SebRoLENS/curvemole.git
cd curvemole
uv sync --locked --group dev
uv run curvemole gui
```

For an editable installation using `pip`:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -e .
curvemole gui
```

### 3.7 Confirming the installed version

```
curvemole --version
```

The About dialog also displays the version. A project stores the application version that created it, and exports include the application version in machine-readable metadata.

### 3.8 Version badge and updates

The status bar contains a clickable version badge. CurveMole checks the latest GitHub release at startup and once per hour while the application remains open. The badge is:

- green when the installed release is current;
- yellow when a bug-fix update (`x.x.y`) is available;
- red when a feature (`x.y.x`) or major (`y.x.x`) update is available;
- grey while checking or when the check cannot be completed.

An automatic notification is shown only once for each newly detected release. Click the badge or choose **Help > Check for updates** to check again, inspect the release notes, or reopen the update dialog.

Linux AppImage and Windows desktop packages can normally use **Update now**. CurveMole downloads the matching release asset and verifies the SHA-256 digest supplied by GitHub. Linux replaces the AppImage, Windows portable installations update from the portable ZIP, and installed Windows applications launch the verified installer. The application directory must be writable and no fit or uncertainty task may be running. Existing Windows portable users are offered a one-time migration to the installer; projects, settings and plugins remain in their existing user directories. Python, source, macOS, unsupported architectures, and protected installations must be updated manually from the release page.

## 4. A complete first fit

This tutorial uses `examples/gaussian.csv` from the source archive. It contains x, y, and `sigma_y` columns.

### 4.1 Import the example

1. Start CurveMole.
2. Choose **File > Import data** or press **Ctrl+I**.
3. Select `examples/gaussian.csv`.
4. In the mapping dialog, confirm comma delimiter, decimal point, and header.
5. Select x as **X column** and check y under **Y column(s)**.
6. Select `sigma_y` as the uncertainty type and column.
7. Press **OK**.

The curve appears in the left tree and becomes active. The central plot displays the data, and the model panel on the right identifies the active curve.

### 4.2 Add a background

1. Press the **+** button in **Model and parameters**, or press **Ctrl++**.
2. Select **Constant** and enable **Add as background**.
3. Keep composition set to `add`.
4. Press **OK**.
5. In the parameter table, enter an initial `offset` near the visual baseline.

### 4.3 Add a Gaussian peak graphically

1. Add another component and select **Gaussian**. Leave **Add as background** and **Initialize from manually selected points on the graph** unchecked.
2. Press **OK**. The pointer changes to a crosshair.
3. Click at the approximate peak center.
4. Keep the mouse button pressed and drag horizontally from the center to either half-height edge.
5. Release the mouse button.

CurveMole uses twice the dragged distance as the initial FWHM. It estimates a positive peak height from the data minus the existing model, converts this height to the area parameterization, and adds the component as one undoable action.

If placement is incorrect, press **Ctrl+Z** and repeat. Press **Esc** during placement to cancel without adding the component.

### 4.4 Adjust the model before fitting

Select the Gaussian in the component list. Three green handles appear:

- drag the central target to change center and height;
- drag either vertical side handle to change FWHM;
- edit exact values or bounds in the parameter table.

For this example, a positive lower bound on `sigma` already exists intrinsically. You may enter a tighter scientifically motivated range, but it is not required.

### 4.5 Mask an interval if needed

Mask editing is active only while the graphical **Mask** toolbar button is enabled:

1. click **Mask**; the button becomes grey while masking is active;
2. in Single view it acts on the displayed spectrum; in Overlay/Waterfall choose the active spectrum or all visible spectra in the scope prompt;
3. right-drag to mask an interval, left-drag to restore it;
4. click **Mask** again to return to normal navigation.

The example does not normally require masking.

### 4.6 Run the fit

1. Press **F5** or choose **Fit > Fit**.
2. Use **Single / independent** mode.
3. Check that the example curve is enabled in the table.
4. Keep **Local least squares (automatic)** and `linear` loss.
5. Press **OK**.

After a successful fit, CurveMole commits the optimized parameter values back into the displayed model and immediately redraws the model sum, individual components, and residual panel. Parameter standard errors appear in the `±1 sigma` column when covariance is available. Open **View > Diagnostics** to inspect residual warnings.

### 4.7 Save and export

Save the complete workspace with **File > Save project as**. Use a `.fitproj` extension.

Then choose **File > Export analysis bundle**. Select a destination and choose both how to save and **What to export?**. The default selection writes only `fit_results.csv`, which associates every curve with its model functions and lists each parameter together with its fitted value and standard error. Data tables, plots, reports, JSON, reusable models, project copies, uncertainty matrices, and diagnostics are optional.

# 5. Main window and controls

## 5.1 Series and curves tree

The left dock contains a search field and a tree with **Visible**, **Series / Curve**, and **State** columns.

- Click a curve to activate it.
- When switching spectra, the function selected in Model & Parameters stays at the same position in the function list, regardless of its name or type. If the new spectrum has fewer functions, its last function is selected; an empty model clears the parameter table. Show all functions retains its project-wide selection.
- Use Ctrl-click or Shift-click for a multi-selection, or use **Select all** and **Deselect all** above the tree.
- Use **Remove selected** to delete one or more accidentally imported curves. Removal is undoable.
- Use **New series** to create an empty named series. Empty series are retained when the project is saved.
- Right-click one or more selected spectra and choose **Move selected to series** to reorganize them.
- Right-click selected spectra and choose **Move selected up/down** to change their order inside a series. Multiple adjacent spectra move as a stable group.
- Right-click a series and choose **Merge series into** to append all its spectra to another series and remove the source series.
- Moving, merging, creating, and reordering series are all Undo/Redo operations and do not invalidate fitted parameters.
- Use the visibility checkbox to include or exclude it from Overlay and Waterfall displays. Hover over the checkbox for an explanation. Hiding a spectrum retains its data and fit; hidden spectra are excluded from fit plans. Highlighting rows separately selects spectra for operations. Hidden spectrum names appear muted; Up/Down navigation skips them. Click a hidden spectrum to inspect it, or check its visibility box to include it in navigation again.
- Double-click an editable name, or right-click and choose **Rename data…** or **Rename series…**. Series names must remain unique.
- Right-click a series and choose **Delete series…** to remove it together with all its curves, models and results. A populated series requires confirmation; Undo restores the complete series. Original files on disk are retained.
- Right-click a **curve** and choose **Choose data colour…** to set its colour. Red is reserved for the fitted Model sum and cannot be assigned to a spectrum.
- Right-click a **series** and choose **Series palette** to recolour the complete series with one of the built-in non-red palettes. Palette and individual colours are saved in the project.
- Use the search field to filter curves by name without removing them.

Curve states are:

| State | Meaning |
|---|---|
| Not fitted | No successful fit is associated with the current curve state |
| Ready | The curve is prepared for fitting |
| Running | A fit involving the curve is in progress |
| Fitted | The last fit completed successfully |
| Modified/outdated | Data or model changed after the last successful fit |
| Failed | The latest fit attempt failed |

Uncertainty tags are added to the fit state for the method currently selected in **Uncertainty Analysis > Method**. A failed replica, an adaptive stability limit or a timeout adds a warning icon beside the spectrum and an amber status. Hover over the State column for the reason and counts. Switching the method updates these tags without refitting or deleting results. It is the same universal method selected through the **Analysis - / +** header in **Model and parameters**: changing either control updates both panels and the spectrum tags. See Section 12.

### 5.2 Plot workspace

The central workspace contains the data/model plot and an optional residual plot. Controls above it provide:

- **Display:** Single, Overlay, or Waterfall;
- **All series / Active series / Selected:** one-click scope for Overlay and Waterfall;
- **X offset** and **Y offset:** display-only Waterfall spacing;
- **Residuals:** show or hide the linked residual panel;
- **Autoscale:** automatically frame the experimental data when the active spectrum changes;
- **All points / Unmasked points:** choose whether automatic framing includes masked data;
- **Plot style:** draw imported data as Lines, Points, or Lines + points;
- **Visual only - background-subtracted:** preview subtraction of marked background functions without changing data or fit inputs.

Choose **View > Plot appearance...** for separate data, function and residual styles, line widths, symbols, colours and grid settings. These display settings are stored with the project.

With Autoscale off, switching spectra preserves the view. **View all** and **View unmasked** explicitly frame the corresponding experimental points; model extrapolations, labels and handles do not affect these limits.

The main toolbar contains the graphical **Mask** button. Its scope is chosen on entry in Overlay/Waterfall; mouse buttons determine the operation.

Offsets never modify data and are removed when converting pointer positions back to scientific coordinates. Fitting and export use the original numeric coordinates, not screen offsets.

The coordinate readout displays the pointer coordinate and the nearest finite point from the active curve. Overlay and Waterfall views use adaptive peak-preserving downsampling and clip drawing to the visible range for dense spectra. Single view keeps the full display resolution. Descending x arrays may be reversed only in the temporary display copy so that fast clipping remains available. Fitting, project storage, and export always use the complete original arrays.

Model functions initially receive names such as **Voigt1**, **Voigt2**, and **Gaussian1**. Right-click a function and choose **Rename function...** to assign any non-empty name, including Unicode. The name belongs to that component of that spectrum; internal identifiers and parameter links do not change. Renaming supports Undo/Redo, preserves fitted status, and survives saving/reopening, duplication and copying. Plot labels, notebook entries, uncertainty tables and exports use the name. Their labels are shown above each function maximum by default and are shifted vertically when necessary to avoid overlap. Right-click the main plot and toggle **Show component labels** to hide or show these labels. After every completed fit, CurveMole explicitly applies the returned optimized parameters to the displayed model and redraws the plot while preserving the current zoom. The **Model sum is always red**; imported spectra and selectable series palettes exclude red so a data curve can never be confused with the fitted sum.

### 5.3 Navigation and axes

Normal plot navigation is available when neither placement nor Mask mode is active. Left double-click frames all experimental data, including masked points. Right double-click frames only unmasked experimental points. Hold the right button and drag to pan the spectrum; a single right-click opens the plot context menu after the double-click interval. Display controls and the coordinate readout wrap onto additional rows when dock resizing narrows the plot. Axes always retain the original data units. Numeric tick labels switch to scientific notation only when their decimal representation is too wide for the available space; this never changes spectrum values or adds an SI unit prefix. The mouse wheel remains available during Quick Add peak placement, spline placement, Quick Fit, and live fit refreshes. A refresh preserves the current viewport instead of resetting a zoom chosen by the user. When the plot has keyboard focus, Up and Down activate the previous or next checked spectrum in project order. Up/Down in the curve tree also skips unchecked spectra.

Use **View > Axes** for:

- automatic range;
- logarithmic x or y;
- reversed x or y;
- locked plot view.

Reversing an axis is only a view setting. Logarithmic axes do not transform the data used by the fit. Values incompatible with a log display may not be visible even though they remain stored.

### 5.4 Model and parameters dock

The right dock normally lists components for the active curve. Enable **Show all functions** to list every model function in the project, grouped by series and spectrum. Group headings cannot be selected as functions. The list supports Ctrl-click and Shift-click multi-selection, including selections across spectra. Mathematical composition order matters because operators are evaluated sequentially. **Reorder** sorts only this displayed list and automatic names; it preserves the composition order and fit. The Up/Down controls explicitly change composition order. Exported parameter rows and component trace columns follow this displayed order and use the current function names. Their values and uncertainties remain associated with the same functions, including models with multiplication, division or convolution.

**Select all** selects every displayed function. **Select all except backgrounds** selects only functions that are not marked as background and clears any previous background selection. In Show all functions mode these actions span the project. Selection is separate from enabling: a checked function contributes to the model and fit; an unchecked function remains stored but is excluded. Hover directly over its checkbox for an explanation. Changing a checkbox on a selected function applies the enabled state to the whole selection. **Mark as background** classifies the selected functions; it does not subtract them from the data.

The controls below the list are:

| Control | Action |
|---|---|
| + | Add a component |
| Duplicate | Duplicate the selected component and its settings |
| Up / Down | Change component order |
| Delete | Delete the component after confirmation |
| Reorder | Sort the function list and renumber automatic names using a parameter value |

| Control | Action |
|---|---|
| Advanced reorder rules | Choose the numbering parameter separately for each function type |
| Copy fit | Copy selected model information to other curves |
| Copy fit to next | Copy to the next curve in the source series, then open it |
| Copy parameter | Copy one parameter from one source function to chosen project-wide targets |

Click the **Analysis - / +** column heading to open **Displayed uncertainty**, a menu of methods recorded for at least one spectrum. The checked entry is the method currently displayed. There is no separate selector above the parameter table. The sole available method is selected automatically. With several methods, choose the one to show for every spectrum: spectra without it show a dash instead of another method's errors. The choice survives switching spectra, Show all functions, saving and reopening. With no recorded analysis, clicking the selector explains how to run one and offers a link to **Uncertainty Analysis**. It remains available when several functions are selected, even while the parameter table is hidden.

Bulk actions on selected functions can delete, duplicate, reorder, enable or disable, mark or unmark backgrounds, and fix or free their parameters as applicable. Each bulk edit is one Undo/Redo operation. Uncheck a component in the list to disable it without deleting it. Disabled components are not evaluated or fitted.

Deleting the first enabled function can leave a multiplication, division or convolution without a preceding term. In that case, the confirmation identifies the first surviving function and explains that it will start the remaining model with **Add**. Other operators and parameter values are preserved. Undo restores the original functions and composition. If remaining parameters link to a function being deleted, deletion is cancelled and the affected links are listed; remove those links or constraints before deleting the function.

**Copy parameter** is enabled only when exactly one source function is selected. Select a row in the parameter table to make that parameter the default, open the copy window, and explicitly check the target functions. The window shows the source spectrum, function, parameter, and numeric value. By default only the value is copied; optional controls also copy bounds, fixed/free state, and the parameter link/relation. Targets without the parameter or whose retained bounds reject the source value are skipped and reported. The complete copy is undoable.

### 5.5 Quick actions

The main toolbar includes a bordered **Quick Add** group with an **Add** button, a function selector and two options. Its compact layout keeps the toolbar at the same height as the other buttons. The selector contains the complete current registry: built-ins, project formulas, reusable user formulas, and loaded plugin functions. CurveMole remembers the last selection across sessions and also updates it when a function is added through the ordinary Add dialog.

The **Background** and **Initialize with points** checkboxes below the selector configure the next Quick Add insertion. The choices are remembered for each function only after explicitly changing them here. Both start off, including for Linear and Cubic spline; the regular Add dialog's choices are independent. Normal peaks use centre/FWHM click-drag placement. Other functions whose point initialization is off are added immediately. During Quick Add, right-click or press Return/Enter to finish, just like **Finish**. Point placement still requires its minimum number of points. Section 8 describes point placement, initial locking and graphical controls.

The toolbar also provides the most common project, model, fitting, background, and calculator actions as icons. Hover over an icon to see its action name.

### 5.6 Dockable tools

The **View** and **Tools** menus show or hide these docks:

- **Worksheet:** read-only view of current x, y, uncertainty, weight, and mask state;
- **Diagnostics:** residual summary and warnings;
- **Log:** operational messages and errors;
- **Data Calculator:** reversible scalar and curve-to-curve transformations;
- **Function Builder:** safe mathematical custom functions;
- **Uncertainty Analysis:** Monte Carlo, bootstrap, and profile calculations;
- **Laboratory notebook:** project notes and searchable object descriptions;
- plugin panels contributed by installed plugins.

Opening a tool places it in the shared tab group beside **Model and parameters** and selects its tab. Closing or hiding a tab preserves the panel and its current state; opening it again returns to the same panel. Tabs can still be moved or floated. When many tabs are open, use the arrows to scroll through their titles. Titles longer than 25 characters end in an ellipsis; hover over one to read its full title. CurveMole remembers window geometry, dock arrangement, and theme. Use **View > Reset layout** to restore the standard tabbed layout.

### 5.7 Themes

**View > Theme** provides System, Light, and Dark themes. Theme choice affects only presentation. It does not change plot data, exported numeric values, or fitting.

### 5.8 Drag and drop

Drop a `.fitproj` file onto the window to open it. Drop any other local file to begin content-aware data import, regardless of its extension. If a drop contains both a project and data files, the first project is opened.

# 6. Importing data

## 6.1 Supported text formats

The graphical importer shows `.txt`, `.dat`, `.csv`, and `.tsv` as common choices but can open any text-based file whose contents form a valid numeric table. For example, spectrometer files named `.xy`, `.xye`, or with a vendor-specific suffix can be selected through **All files** or dropped onto the window. The extension is not used as proof of validity. Files must contain at least two columns and at least two rows after parsing.

CurveMole detects common delimiters:

- whitespace;
- comma;
- semicolon;
- tab;
- vertical bar.

It also proposes decimal point or decimal comma and whether the first data row is a header. Always inspect the preview before accepting the mapping.

## 6.2 Comments, blank lines, and encoding

Blank lines are ignored. Lines whose first non-space character is `#`, `;`, `%`, or `!` are treated as comments by the default importer. UTF-8 with an optional byte order mark is the default encoding.

CurveMole also detects leading instrument metadata or preamble lines before the rectangular table. **Ignore first rows** shows the proposed number of physical lines to skip and can be changed from 0 to 1,000,000. Skipping occurs before delimiter, decimal, and header interpretation; therefore a column-name row after the preamble can still be used as the first data-row header. The accepted value is stored in the curve import metadata for reproducibility.

The semicolon can therefore be ambiguous: a line beginning with semicolon is a comment, while semicolons inside table rows can be delimiters. Check the preview for locale-specific exports.

Before confirming files, the file chooser already includes a small spectrum graph. Highlight a file to preview it; select X and Y from the adjacent dropdowns (first and second column by default). Reading happens in the background. The graph uses the same peak-preserving adaptive renderer as the main plot and full-data axis limits. Choices are remembered per file and proposed in the subsequent import mapping dialog. With multiple files, Apply this mapping to all files deliberately uses the mapping confirmed for the first file. Files requiring custom parsing can still be opened when the quick preview is unavailable, then configured in the mapping dialog.

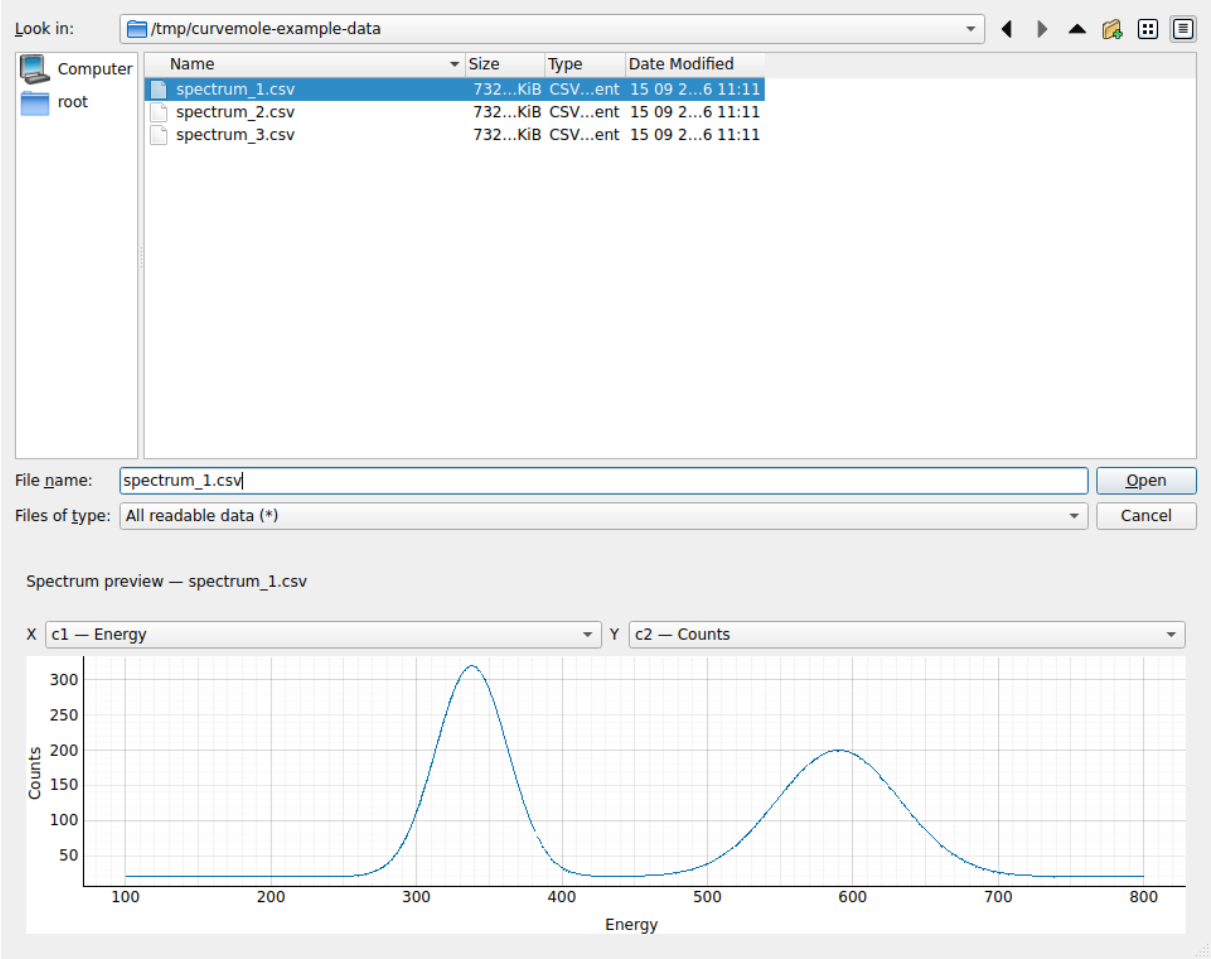



Figure 1: Preview while selecting files, before opening the import mapping dialog.

### 6.3 Column mapping

Ordinary imports select one X and one active Y. All original columns remain available in the imported curve. For files with more than two columns, the **as separated files** checkbox appears directly below the Y selection. Hover over it for an explanation. Enabling it imports independent spectra in the same series, named after their Y headers (or `Y - column N` without headers); it does not create files on disk.

Choose the structure explicitly:

- **Shared X:** check multiple Y columns, all using the selected X. **Select all Y columns** selects every column except X; deselect any error or auxiliary columns before importing.
- **Repeated groups:** choose the first column, number of Ys per X, and error type per Y, then click **Apply pattern**. One Y per X produces `XYXY...`; two produces `XYYXYY...`. With symmetric errors these become `XYE...` or `XYEYE...`; asymmetric errors use `YE+E-` for each Y. Review or edit the resulting associations. Incomplete trailing spectra are excluded, and the unassigned column numbers are shown below the table.
- **Custom:** add or remove spectrum rows and choose any X/Y associations, including nonadjacent columns and irregular groups. Switching from repeated groups to Custom retains the generated rows for editing.

Column names never infer the structure, axes, or error columns. Each table row has its own error/weight type, error columns, and confidence level. When an error type is enabled, its selector initially uses the column immediately to the right of Y; asymmetric errors select the next two columns as error + and error -, respectively. These defaults can be changed. Missing columns or errors overlapping an assigned X/Y must be resolved before importing. Error types initially remain **None**. The same adjacent-column defaults apply to ordinary imports when errors are enabled.

Optional columns are:

- `sigma_x`;
- `sigma_y`;
- Y error (confidence interval), or a pair of asymmetric Y error columns;
- generic weight;
- variance;
- inverse variance.

Only one of `sigma_y`, confidence-interval errors, generic weight, variance, or inverse variance can be selected for a given spectrum. Variance is converted to `sigma_y` by square root. `sigma_x` is stored in the project but is not used by the version 0.42.0 optimizer.

For confidence intervals, choose **Y error (confidence interval)** and one symmetric half-width column, or check **Asymmetric error** and select **Y error +** and **Y error -** separately. Both columns contain positive magnitudes, not signed errors or interval endpoints: the interval is `[y - error -, y + error +]`. **Data error confidence** declares its central coverage (default 95%). This is independent of the confidence selected for fitted parameter intervals. Existing `sigma_y` imports continue to mean one standard deviation, regardless of this field. The import preview shows the supplied errors at their original confidence.

The spectrum preview below the numeric table updates as you change X/Y columns, uncertainty mappings, delimiter, decimal separator, header or skipped rows. It plots the complete file, reusing the main graph's peak-preserving downsampling and clipping rules. Parsed data are cached while changing columns. Descending X is reversed for display; nonmonotonic X is not clipped/downsampled. The view bounds always come from full-resolution finite X/Y pairs. With no selected Y or an invalid mapping, the preview clears and explains the problem. For batch import, it previews the file named in the mapping dialog.

## 6.4 Batch import

When importing multiple files, **Apply this mapping to all files in this batch** reuses the first file's parsing and column mapping. Enable it only if all files truly share a layout. CurveMole does not silently guess a new mapping for an incompatible file in the same batch.

Separated-spectrum mappings reuse column positions, so files with the same layout can have different headers. Each spectrum keeps the Y name from its own file. Ordinary batch mappings continue to use the selected column names.

Files are imported in natural filename order: `scan_2` precedes `scan_10`, regardless of the order returned by the file chooser.

### Calibrated WinSpec SPE files

CurveMole also reads binary WinSpec SPE 2.x files containing calibrated one-dimensional acquisitions. The first column is wavelength in nm, followed by one intensity column per frame. Select the frame through the Y-column mapping. SPE 3.x, uncalibrated axes, images and multiple-ROI acquisitions are unsupported; export those acquisitions to calibrated x-y text before importing.

### Automatic folder import

Choose **File > Automatic folder import...** to add arriving files to the current project. Select an existing folder, an optional case-insensitive **Filename contains** filter, the one-based X/Y columns and a **Wait for stable file** delay (2 seconds by default). Monitoring starts only when you press **Start**.

- **Also import matching files already in the folder** includes existing files; otherwise the session starts with newly arriving files.
- **Import changed files again as NEW spectra** retains the previous acquisition and adds a new one when file contents change.
- **Show newest imported spectrum** controls whether successful imports take focus; it can be changed while monitoring runs.
- **Automatic workflow** selects Import only or a loaded plugin's import processor. Configure the plugin first; processing runs on the new acquisition in the background before it is added to the live project.

Only files directly in the folder are scanned. Symlinks, hidden/temporary files and projects are ignored, and automatic inputs are limited to 256 MiB. Identical content at the same path is not imported twice within a session. Each successful acquisition is one Undo operation. Failed files appear in the log; correct them or the settings and choose **Retry failed files**. Imports wait while fitting, a desktop automation or a modal dialog is active. Closing the monitor panel leaves it running; choose **Stop** or **File > Stop automatic import** to stop. Changing projects ends the session, and reopening CurveMole does not start monitoring automatically.

## 6.5 Invalid numeric cells

Non-numeric cells are converted to missing numeric values. Rows are retained in the stored arrays but excluded from fitting when x or y is not finite. A row is also excluded if:

- `sigma_y` is non-finite or not strictly positive;
- a point weight is non-finite or negative;

- it is masked;
- it lies outside active fit ranges defined through the core API.

CurveMole does not reorder rows, average duplicate x values, or interpolate missing values during import.

### 6.6 Uncertainties and weights

If absolute `sigma_y` values are supplied, point residuals are scaled as:

$$r_i^{(w)} = \frac{f(x_i) - y_i}{\sigma_{y,i}}.$$

If inverse variances $w_i = 1/\sigma_i^2$ are supplied, CurveMole multiplies residuals by $\sqrt{w_i}$. A generic weight column directly multiplies residuals by its value. This distinction is intentional and is recorded in the curve metadata.

Zero generic weight is allowed and gives a point zero numerical influence. A zero or negative `sigma_y` is invalid and excludes the row.

For a confidence level $C$, interval half-widths are converted to approximate Gaussian scales by dividing each side by $z = \Phi^{-1}((1 + C)/2)$. At every evaluation, a prediction above the measured value uses the upper scale; a prediction below uses the lower scale. Equal half-widths reproduce symmetric Gaussian weighting. The error arrays and confidence are saved and exported; plotted bars keep the supplied half-widths, rather than these converted scales.

**Fit > Advanced algorithm options > Use data errors / weights in the fit** is on by default. Turn it off to fit without imported Y errors or point weights. Masks, fit ranges and spectrum-level weights remain in effect. Rows invalid only because of their error or point weight can then participate. The display of bars and the noise model of Monte Carlo remain independent of this choice.

This is an approximation: two interval half-widths and a confidence level do not specify the full measurement distribution. The scales describe a Gaussian side on each side of the reported value; Monte Carlo assigns equal probability to the two sides, reproducing the supplied central interval. Covariance is a local Jacobian approximation, and asymmetrical parameter intervals should be assessed using resampling or a suitably calibrated profile. Correlation between measured points and X uncertainty are not incorporated into this Y-residual fit.

Use **Error bars** above the main graph to show or hide the bars. Autoscaling includes their endpoints, and bars follow logarithmic, waterfall and background views. Scalar and curve scaling propagate widths and exchange upper/lower sides for negative factors. Custom and column formulas use local derivative propagation; for strongly nonlinear transformations this is only a local approximation.

# 7. Curves, masks, and transformations

## 7.1 Immutable originals

CurveMole preserves imported x, y, uncertainty, and weight arrays. Displayed working arrays are reconstructed by applying a transformation history to the originals. This design makes data preparation reversible and auditable.

## 7.2 Masking points and intervals

Masked points remain stored but are excluded from fitting. They are drawn as faded markers, and masked intervals are shaded.

Enable the graphical **Mask** button: right-drag masks an interval and left-drag unmasks it. A right/left click performs the equivalent operation on a point. The button becomes grey while active. Turn it off to navigate without changing exclusions. Changing display mode also exits masking.

## 7.3 Mask targets

| Target | Effect |
|---|---|
| Active | Changes only the active curve |
| All visible | Changes every currently visible curve |

Single view always uses the active spectrum. Overlay and Waterfall prompt for Active or All visible when entering Mask mode; cancelling leaves masking off.

For interval masks, the same numeric x interval is applied to every target. For a single-point mask transferred from the active curve, CurveMole locates the nearest x value on other targets and applies it only if it lies within the configured tolerance.

Set this tolerance through **Data > Mask transfer tolerance**. A value of zero requires an exact matching x value on transferred point masks.

## 7.4 Undo and mask history

A graphical mask action is placed on the application Undo stack. Press **Ctrl+Z** to undo it or **Ctrl+Shift+Z** on platforms that use that standard Redo binding. The project also stores mask arrays and numeric mask intervals.

Fits, including Quick Fit and sequential fitting, also support Undo/Redo. Undo restores the pre-fit models, parameter values, fit results, and sequential pause state; Redo restores the completed fit without running the solver again. Cancelled or failed worker operations restore the previous state. The Undo stack retains the latest 20 operations (fits and other edits combined); input data arrays are not duplicated for fit history. Undo/Redo are disabled while a worker is running.

Quick Fit is available before opening Fit...: its first run uses the default independent-fit settings on the selected curves, falling back to the active curve. Later runs retain the

configured solver, loss and algorithm settings but always use **Single/Independent**, even after a sequential or global fit. Open Fit… to use another fitting mode.

## 7.5 Data Calculator

Open **Tools > Data Calculator**. The following scalar operations are available:

- add to or subtract from y;
- multiply or divide y;
- shift x;
- scale x;
- normalize y by maximum absolute value;
- normalize y by signed integrated area.

Every calculator operation (including restoring data) offers **Active curve** or **Choose multiple spectra…**. The picker initially shows the active series; **All series** shows every spectrum grouped under colored series headings. Use checkboxes or Select all shown. Hidden spectra are excluded. Cancel changes nothing. A batch is checked in full before any spectrum is modified; one Undo restores the whole batch. Normalization also scales `sigma_y` consistently. Dividing by zero and normalizing a zero or invalid signal are rejected.

### Advanced column formulas

The green **Advanced - column formula** entry is first in the operation list, followed by the green **Custom formula** for a single plotted axis. New imports retain every source column as numeric data in the project, including columns not plotted. Original column order and row alignment are preserved. `c1`, `c2`, `c3`, etc. refer to one-based source columns; `x` and `y` refer to the current plotted axes. **Insert into formula** lists the available codes with original column names; choose an entry, then press **Insert** to add its code at the cursor. **Write result to** chooses X, Y or any retained column for the result. The hint below it explains that Apply writes the formula result into this column. Merely selecting an input column does not edit the formula or change its destination. A live summary such as `Y - Counts <- c2 / c3**2` makes the direction explicit. **Spectra to modify** separately chooses which spectra receive the operation.

For a file with Energy, Counts and Monitor columns, plotting c1 against c2:

1. choose **Advanced - column formula**;
2. choose **Y - Counts** as the destination;
3. insert `c2` and `c3` from the dropdown to enter `c2 / c3**2`;
4. choose Active curve or multiple spectra, then Apply non-destructively.

You can also enter `y / c3^2` (`^` and `**` both mean exponentiation in Advanced), `c2 - c4`, or `sqrt(c5)`. A header may be referenced as `${Monitor counts}`; codes remain unambiguous for unusual names. Only restricted mathematical expressions are accepted, never Python execution. Scalar results are broadcast to every row; missing columns and non-finite output (including division by zero) reject the batch. Masks do not change row alignment: formulas operate on all rows.

Writing to a plotted source column updates that axis. Writing to an unplotted column leaves the graph unchanged and makes the new values available to later formulas. Normal calculator operations on X/Y are also reflected in their mapped columns. Calculations are local to each spectrum, even when spectra originated in the same file. In a multi-spectrum batch `cN` always means position N in each file; check that the files have compatible layouts. Uncertainty propagation is not inferred for custom formulas; changing an auxiliary uncertainty column does not remap the uncertainty arrays originally selected for fitting.

Name and **Save formula** to retain the expression and destination in the project, then recall it through Saved formulas. Retained columns, formulas and their Undo/Redo history

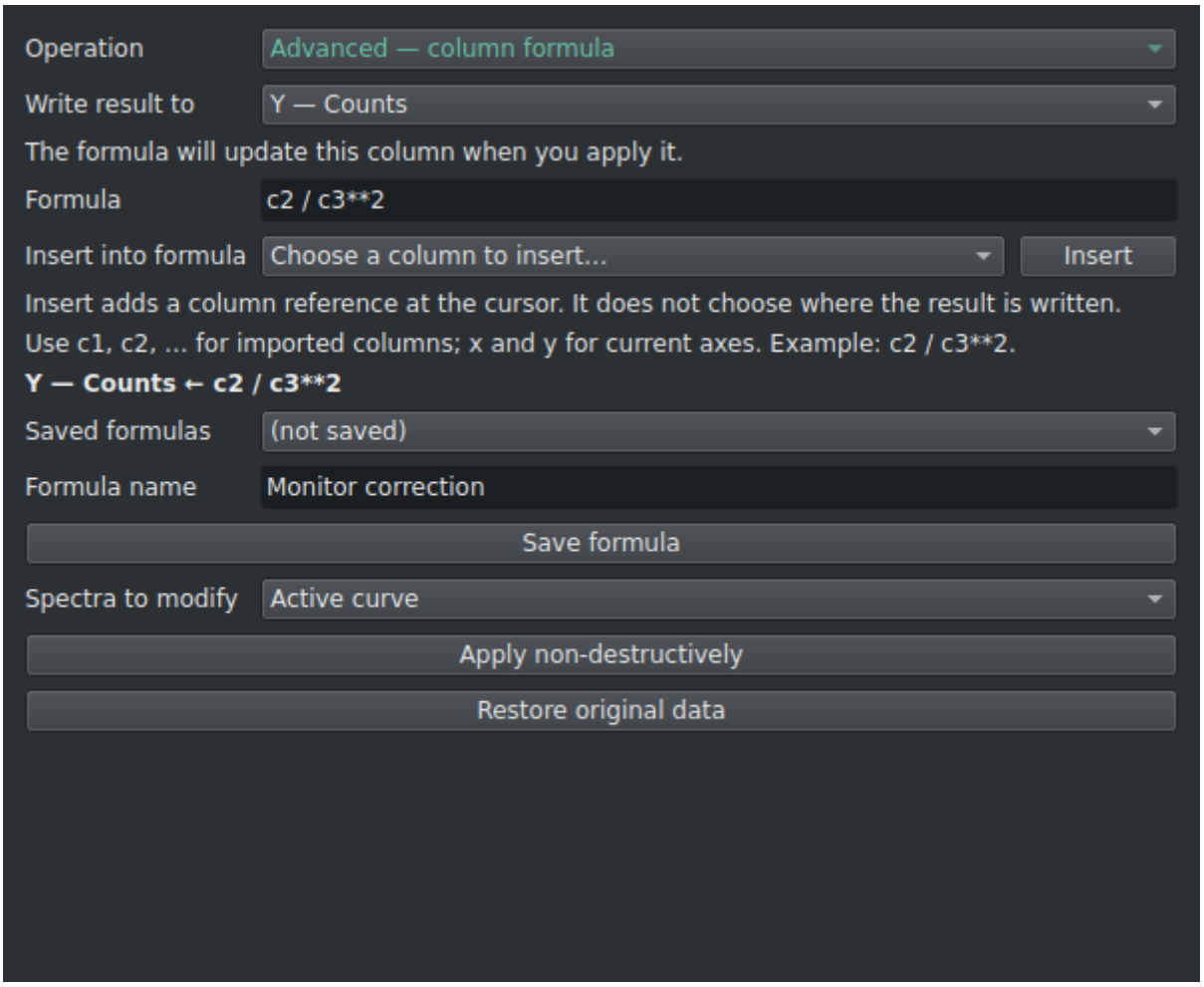



Figure 1: Advanced calculator with column insertion and destination controls in dark mode.

survive `.fitproj` saving without the original source file. Projects created before this feature contain only their previously stored axes: re-import the source to recover additional columns.

### 7.6 Curve-to-curve calculations

The calculator can add, subtract, multiply, or divide by another curve. The operand is aligned to the target x grid by:

- linear interpolation;
- nearest-neighbor interpolation;
- cubic spline interpolation.

Extrapolation is disabled by default. Outside the operand range, aligned values become missing and will therefore be excluded from later fitting. Enabling extrapolation is an explicit advanced choice. Cubic interpolation requires at least four unique finite operand x values.

The aligned operand array is stored with the transformation so that reopening the project does not depend on a later change to the source curve.

### 7.7 Restoring original data

**Restore original data** removes all currently applied transformations from the selected target while retaining them on the redo stack. The imported arrays are not re-read from disk. The whole action is also integrated with application Undo.

### 7.8 Worksheet

The Worksheet is a read-only inspection view. It shows up to 100,000 rows for the active curve with columns for x, y, `sigma_y`, weight, and mask state. It is not a spreadsheet editor and does not modify source arrays.

### 7.9 Editing individual data points

Choose **Data > Edit data points...** to edit measured points directly in the graph. The editing controls appear above the plot. Only the active, displayed dataset is edited; the original source coordinates remain immutable.

1. Click a point to select it. Its current X/Y coordinates appear in the editing bar.
2. Choose **X**, **Y**, or **XY** to constrain the data coordinates that may change.
3. Drag the point and release to commit one undoable move. During dragging, the graph previews the new position without changing the saved dataset.
4. Use **Delete selected point**, **Delete**, or **Backspace** to remove it. At least two points must remain. Deletion removes the row from current data, fitting, worksheets and numeric exports, keeping associated errors, weights, masks and auxiliary columns aligned.
5. Choose **Done** or press **Esc** to leave editing. Esc also discards an unfinished drag; previously completed moves and deletions remain applied.

Right-drag pans and the wheel zooms while editing. Point selection uses the actual samples, including in dense plots. Logarithmic axes, Waterfall offsets and visual background subtraction are accounted for; the editing bar shows the underlying data coordinates. X/Y constraints refer to those data coordinates.

Moves and deletions use the global **Undo/Redo** commands. They are stored in the project as reversible transformations and can also be removed with **Restore original data** in the Data Calculator. A changed fitted dataset is marked as modified, so refit it to update results and uncertainties. Entering function placement, changing the active dataset or starting a calculation ends point editing. Read-only projects cannot be edited.

## 8. Building models

All entries in the function library are fitting functions. Constant, linear, polynomial, cubic-spline, peak-shaped, and custom formulas are not separated into a special background-function class. Background is instead a property of a model component. Select a component in **Model and parameters** and enable **Mark as background** when that component represents the experimental baseline or background.

**Data > Subtract background...** uses these component-level marks. If no component is marked, CurveMole first asks which model functions should be designated as background. If background components already exist, it asks which of the marked components should be subtracted. The subtraction uses the current resolved/fitted parameter values, is reversible with Undo, applies over the complete data array, and disables the subtracted components afterwards to prevent double-counting.

The **Subtracted / Not subtracted** labels follow the subtraction history of each spectrum. Copying a fit preserves the destination's existing subtraction; it does not subtract the source background from the destination's data. The saved subtraction retains the parameter values used at that time. Editing or copying later parameter values does not recalculate it. A duplicated function starts with its own history and is not marked as subtracted merely because its source was. On opening older projects, CurveMole can reconnect a recorded subtraction to a uniquely identified builtin background whose internal ID changed. This restores the label and function-state restoration without changing the measured data, the stored subtraction array or the current fitted parameter values. If the old history is ambiguous, the label reads **Subtraction recorded - function uncertain**; **Revert background** still uses the actual recorded array. Confirming Revert removes the subtraction, so **Not subtracted** is then the expected status.

The same dialog can apply the operation to every eligible spectrum, using the enabled functions already marked as background in each model. **Data > Revert background...** lists only spectra changed by this dedicated subtraction command and restores the selected ones, including the relevant function states. **Visual only - background-subtracted** instead removes the enabled marked background only from the plot; it never changes curve arrays, fit inputs, or exported values.

During graphical cubic-spline placement, nodes may be placed anywhere in plot coordinates, including outside the x/y extent of the measured data. Adding nodes does not autorange the graph. Left-drag continues to pan and the mouse wheel continues to zoom while placement is active.

### 8.1 Component composition

Components are evaluated in list order. Available operators are:

| Operator | Operation |
|---|---|
| `add` | Add the component array to the accumulated model |
| `subtract` | Subtract the component array |
| `multiply` | Multiply the accumulated model by the component array |
| `divide` | Divide the accumulated model by the component array |
| `convolve` | Convolve using FFT and scale by median x spacing |

The first enabled component can use only `add` or `subtract`; multiplication, division, or convolution requires an existing accumulated model. Additive peak plus background models should normally use `add` for every component.

Division by a component that reaches zero and other non-finite model evaluations are rejected by the fitter.

## 8.2 Built-in peak parameterization

Every built-in peak uses signed integrated `area`, not peak height, as its intensity parameter. Negative areas produce negative peaks. This common parameterization makes areas comparable between different line shapes.

### Gaussian

Parameters: `area`, `center`, `sigma`.

$$G(x) = \frac{A}{\sigma\sqrt{2\pi}} \exp\left[-\frac{1}{2}\left(\frac{x - x_0}{\sigma}\right)^2\right].$$

The derived FWHM is $2.354820045\,\sigma$. `sigma` is intrinsically positive.

### Lorentzian

Parameters: `area`, `center`, `gamma`.

$$L(x) = \frac{A\gamma}{\pi[(x - x_0)^2 + \gamma^2]}.$$

`gamma` is the half width at half maximum, so FWHM is $2\gamma$. `gamma` is intrinsically positive.

### Voigt

Parameters: `area`, `center`, `sigma`, `gamma`.

The Voigt component is the area-normalized convolution of a Gaussian with standard deviation `sigma` and a Lorentzian with HWHM `gamma`. CurveMole evaluates it with the SciPy Voigt profile. Both width parameters are intrinsically positive.

The displayed FWHM is the approximation:

$$\text{FWHM} \approx 0.5346(2\gamma) + \sqrt{0.2166(2\gamma)^2 + (2.354820045\,\sigma)^2}.$$

### Pseudo-Voigt

Parameters: `area`, `center`, `fwhm`, `eta`.

The profile is an area-normalized linear mixture with a common FWHM:

$$pV(x) = A[(1 - \eta)G_{\text{unit}}(x) + \eta L_{\text{unit}}(x)].$$

`fwhm` is intrinsically positive and `eta` is intrinsically bounded to $[0, 1]$. `eta=0` is Gaussian and `eta=1` is Lorentzian.

### 8.3 Built-in functions commonly used as backgrounds

The library names are **Constant**, **Linear**, **Polynomial** and **Cubic spline**. They become background components only when **Add as background** in the Add dialog or **Mark as background** in the model panel is enabled.

#### Constant

Parameter: `offset`.

$$B(x) = c.$$

#### Linear

Parameters: `intercept`, `slope`.

$$B(x) = c_0 + c_1 x.$$

#### Polynomial

Choose an order from 0 to 50 when adding the component. Parameters are `c0`, `c1`, and so on through the chosen order:

$$B(x) = \sum_{k=0}^{n} c_k x^k.$$

High polynomial orders are numerically sensitive when x values are large or poorly scaled. Prefer a low order or rescale x with the Data Calculator when scientifically appropriate.

#### Cubic spline

Spline x nodes are metadata, rather than optimizer parameters. Their y values (`y0`, `y1`, and so on) are normal parameters that may be fitted, fixed, bounded, or linked. Two nodes produce linear interpolation. Three or more nodes produce a natural cubic spline, with extrapolation outside the node range.

New graphically placed spline y nodes are fixed by default so an intentionally drawn baseline does not move when fitting begins. Uncheck **Fixed** for individual nodes, or use **Lock all** / **Unlock all** below the parameter table for every parameter in the selected function. The spline remains continuous through masked intervals even though masked measurements do not contribute to the objective function.

### 8.4 Quick Add Function

Choose a registered function from the selector inside the **Quick Add** toolbar group and press its **Add** button. The list includes built-in peaks and backgrounds, generic functions, project-contained formulas, reusable formulas, and trusted plugin functions. The last choice is remembered. Below the selector, check **Background** to mark the added function as background and **Initialize with points** to initialize it from graph points. Both options default to off. Clearing point initialization uses the function's normal insertion mode.

- a peak with manual-point initialization off allows repeated centre/FWHM click-drags until **Finish**, right-click or Return/Enter ends Quick Add;
- a cubic spline with **Initialize with points** enabled opens point-by-point node placement;
- Linear and other functions use manual-point placement only when explicitly enabled using **Initialize with points**;
- functions with point initialization off are added with their initial parameters; a spline in this mode uses an initial node grid with free y parameters.

Quick Add therefore uses the same registry and initialization logic as the normal Add dialog. Selecting a function in the normal Add dialog also makes it the next Quick Add choice. Quick Add remembers its own manual-point and background options separately for each function. It does not inherit options from the normal Add dialog or its older shared preferences. In Quick Add point placement, right-click and Return/Enter commit the same points as **Finish**, provided the minimum has been reached; otherwise placement stays active. **Undo point** removes the most recent point. In the regular Add dialog's point placement, right-click continues to remove the nearest point.

### 8.5 Adding a peak with the pointer

After choosing a peak and pressing **OK**:

1. click the intended center;
2. drag horizontally to either half-height edge;
3. release to set the initial full width.

A green region previews the width. If no horizontal width is selected, CurveMole uses a default based on 5 percent of the x span, local point spacing, and floating point precision.

For built-in peaks, CurveMole converts the graphical height and FWHM into the native area and width parameters. New custom peaks should declare explicit semantic roles for their position, intensity, and width parameters in Function Builder. Older custom functions without role metadata retain compatibility through conventional names such as `center`, `x0`, `fwhm`, `sigma`, `gamma`, `area`, `amplitude`, or `height`.

### 8.6 Initializing any function from manually selected points

In Add component, enable **Initialize from manually selected points on the graph**. This is selected by default for Linear and Cubic spline and can be used for other built-in, custom or plugin functions. For a custom formula, Function Builder can set **Use manual-point initialization by default**.

Click explicit points representing the function you want to create. A live green function preview updates as points are added or removed; it is an initial estimate, not a fit to the measured spectrum. Linear requires at least two points. Other non-spline functions require at least two points and at least as many points as parameters. Cubic spline requires at least two distinct x positions.

Use **Finish** or double-click to commit after the minimum number of points; **Undo point** removes the most recent point, right-click removes the nearest point, and **Esc** cancels insertion. The resulting parameters are fixed by default. Use **Unlock all** or individual **Fixed** checkboxes before optimizing them. Points and the corresponding controls are stored with the component and project.

#### Drawing a cubic-spline background

After choosing **Cubic spline**, enable **Add as background** and keep manual-point initialization enabled:

1. left-click background nodes on the plot;
2. inspect the live green preview, which updates after each click;
3. use **Undo point** if necessary;
4. finish with **Finish** or double-click after at least two nodes; right-click removes the nearest node;
5. press **Esc** to cancel the entire placement.

Nodes are sorted by x. Clicking again at effectively the same x updates that node instead of creating a duplicate. Exact duplicate x positions are not permitted.

### 8.7 Graphical editing after placement

Select a peak component to display its handles. Drag the central target to change center and area-derived height. Drag a side line to change FWHM. For a Voigt peak, changing FWHM graphically scales `sigma` and `gamma` by the same ratio, preserving their current relative contribution.

For a function created from manual points, selecting it displays its original control points. Unlock the relevant parameters before dragging a point in x and y. For a spline, unlock that node's y parameter: dragging also updates its stored x position and keeps nodes sorted. For other functions, at least one parameter must be free; CurveMole refines those free values against the moved control points while retaining fixed and linked values. Each completed drag is undoable. Exact numeric parameter edits are available in the table.

For normal peak centre/width handles, fixed values cannot normally be changed by a drag. Hold **Ctrl** while dragging to change the stored value while leaving the parameter fixed. Manual-point handles instead require unlocking the relevant parameters. Linked values cannot be dragged independently because an expression controls them.

### 8.8 Automatic peak suggestions

Choose **Model > Find positive peaks...**. Despite the menu label, the first dialog can search positive, negative, or both signs. The next dialog selects any registered function classified as a peak, initially preferring the current Quick Add peak. CurveMole then:

- subtracts the median as an initial baseline;
- estimates noise through a robust median absolute deviation;
- uses a default prominence of the larger of three noise estimates or 1 percent of the y range;
- estimates width at half prominence;
- sorts suggestions by prominence.

Choose how many suggestions to add. Every suggestion is initialized using the selected peak function's explicit parameter roles or, for legacy functions, recognized parameter names. Suggestions are initial estimates, not a scientific decision about the number, identity, or line shape of peaks.

### 8.9 Duplicating, disabling, deleting, and reordering

Duplicate a component to create an adjacent copy with a new internal identifier. Disable a component to compare models without losing its settings. Deletion and reordering are undoable. Reordering can materially change models that use operations other than addition or subtraction.

The **Reorder** button sorts the function list in ascending parameter order (peak `center`, or X position, by default), including peaks of different types. It also renumbers automatically named functions within each type to match that order. IDs, links and custom names are preserved; custom-named functions are sorted too. Functions without a finite value for their sorting parameter keep their list positions, and equal values retain their relative order. The adjacent **Advanced reorder rules...** button selects another sorting and numbering parameter for each function type and remembers those rules. Both the list order and automatic names are restored by Undo and reapplied by Redo. **Reorder changes presentation only:** parameter values, links, fit results and the mathematical composition order remain unchanged, including multiplication, division and convolution. The display order is saved with the project and copied with the model. Use the Up/Down controls to change the mathematical composition order explicitly; those controls restore the list to composition order.

### 8.10 Copying a fit to other curves

**Model > Copy fit** copies selected information from the active curve. Use **Select all** or **Deselect all** when choosing many target curves. The dialog initially shows **Active series**; **All series** exposes the entire project, grouped under colored series headings. Only targets in the displayed scope are copied.

**Copy fit to next**, beside Copy fit, opens the same options with the next spectrum in the source series already checked. After a successful copy, that spectrum opens automatically in the plot and parameter panel. Cancelling the dialog leaves the active spectrum unchanged. The action is unavailable at the end of the series. Copying remains undoable. Options include:

- component structure;
- current or best values;
- bounds and fixed states;
- internal links;
- background components;
- masks;
- fit ranges.

Masks and fit ranges are excluded by default because x grids and excluded regions may not correspond scientifically. When copying masks, point transfer uses the configured x tolerance. Review every target model after copying.

# 9. Parameters, constraints, and links

## 9.1 Parameter table

The table columns are:

| Column | Meaning |
|---|---|
| Parameter | Parameter name; parameters with a relation show its indicator |
| Value | Current value or successful fitted value |
| ±1 sigma | Covariance-based standard error when available |
| Analysis - / + | Lower/upper distances from this spectrum's saved result for the project-wide display method; click this heading to choose a method, with a dash if its interval is unavailable |
| Fixed | Exclude the parameter from optimization |
| Lower | Lower bound; blank means negative infinity |
| Upper | Upper bound; blank means positive infinity |
| Link | Graphical **Set link...** control; the relation and its source are shown in readable form |

Press **Set link...** for the target parameter. Its name appears beside the expression: for a centre link, the dialog shows `center =` followed by the expression field. **Equal to source** is the default. The field shows the selected source explicitly and is read-only for quick relationships; **Advanced expression** makes it editable. Use **Remove link** to remove the relation and make the parameter independent again.

### Choose a source spectrum

The **Source spectrum** menu starts with **Same spectrum as this parameter**. Below it, **Specific spectrum (fixed when copied)** groups spectra under the same series headings and in the same order as the project's spectrum list. Each choice explains what happens when the functions are copied:

| Source spectrum choice | After copying to another spectrum |
|---|---|
| **Same spectrum as this parameter** (default) | Use the corresponding source function in the destination spectrum |
| A spectrum under **Specific spectrum (fixed when copied)** | Keep referring to that exact spectrum, even if it is the current spectrum |

For example, link Gaussian2's `center` to Gaussian1's `center` in Spectrum 1. After copying the functions to Spectrum 2, **Same spectrum as this parameter** uses Gaussian1 in Spectrum 2. Selecting Spectrum 1 explicitly under the fixed spectra instead keeps Gaussian2 in Spectrum 2 linked to Gaussian1 in Spectrum 1. Copy behavior is saved for each inserted reference, so one advanced expression can combine relative and fixed references without losing their meaning. Undo/Redo, parameter copying and sequential model propagation preserve those choices.

#### Build an expression from several parameters

In **Advanced expression**, **Source function** and **Source parameter** select a parameter to insert. Click **Add** to insert its readable reference at the expression cursor, then type arithmetic operators, numbers and parentheses as needed. You can insert any number of references and use the same parameter more than once. Selecting another spectrum, function or parameter does not change references already inserted; click **Add** to insert the new selection.

You can also type `(Gaussian2.center + Gaussian3.center) / 2` directly, with or without `${...}` around individual references. New names use the selected **Source spectrum**; once recognized, each reference keeps that spectrum and copy behavior when the selectors change. Use `${Left peak.center}` for function names with spaces, or `${Series / Spectrum / Function.parameter}` for a fixed spectrum reference. Unknown names and duplicate matching names are rejected with a readable message; use **Add** to choose the exact source when names are ambiguous. Typed references are saved with stable identities, just like references inserted with Add.

To set Gaussian1's `center` to the mean of Gaussian2's and Gaussian3's centres:

1. open **Set link...** for Gaussian1's `center` and choose **Advanced expression**;
2. keep **Same spectrum as this parameter** and type `(` in the expression;
3. select Gaussian2, select `center`, and click **Add**;
4. type `+`, select Gaussian3 and `center`, and click **Add** again;
5. type `) / 2` and check the displayed `center =` relation before accepting.

The resulting relation reads `(Gaussian2.center + Gaussian3.center) / 2`, with each parameter shown as an inserted reference. CurveMole retains its stable identity even if the function is later renamed or the list is reordered. References to other spectra also show their spectrum context, so equal names in different spectra remain distinguishable. Existing expressions using `${source}` remain supported, but **Add** lets you build a multi-parameter expression without typing internal identifiers.

#### Choose a quick relationship

Quick relationships use the selected source parameter and show the resulting relation explicitly:

| Relationship | Meaning for the target parameter |
|---|---|
| **Equal to source** | `target = source`; the target is derived from the source |
| **At least source** | `target >= source`; the target remains a free fitted parameter |
| **At most source** | `target <= source`; the target remains a free fitted parameter |
| **Similar to source** | `source - tolerance <= target <= source + tolerance`; the target remains free within this interval |
| **Advanced expression** | `target = expression`; the target is derived from all references in the expression |

For **Similar to source**, enter a positive tolerance in parameter units or as a percentage. An absolute tolerance of `2` means an interval from `source - 2` to `source + 2`. A percentage tolerance of `5` uses `0.05 * abs(source)`, including when the source value is negative. Percentage tolerances must be positive and less than 100.

The source-dependent bounds in **At least**, **At most**, and **Similar** are hard constraints, recalculated throughout fitting as the source changes. They are combined with the target's

ordinary Lower/Upper cells and intrinsic function bounds; they do not replace these static limits or impose a soft penalty. The target remains an independent optimizer variable within the allowed interval, while **Equal to source** and **Advanced expression** remove it from the free parameter count. If the allowed interval collapses to a single value, mark the parameter **Fixed** or use **Equal to source**. This also applies to a percentage tolerance around a source fixed at zero, whose allowed interval is `[0, 0]`.

Relations between different spectra require **Global simultaneous** fitting with all referenced spectra included. CurveMole rejects dependency cycles, including cycles involving source-dependent bounds. An edit is validated immediately: invalid expressions, missing parameters, inconsistent bounds and infeasible constraints are reported explicitly. If a constraint becomes infeasible during fitting, the fit reports an error rather than silently ignoring the relation.

### 9.2 Parameter states

A parameter can be free to fit, fixed, or derived by an equality/expression link. Free parameters can also have lower, upper or interval bounds. Source-dependent **At least**, **At most**, and **Similar** relations add dynamic bounds while keeping the target free; an equality or advanced expression instead derives its value and removes it from the independent optimizer variables. Bounds can be combined with these relationships, and all active limits must be satisfied.

Intrinsic function bounds always remain active. For example, clearing the visible lower cell cannot make a Gaussian `sigma` negative because the function definition sets a positive intrinsic minimum.

### 9.3 Parameter paths

The canonical path is:

```
curve_id.component_id.parameter_name
```

A link refers to a path inside `${...}`. For example:

```
${curve_ab12.component_cd34.center}
```

or a derived relation:

```
${curve_ab12.component_cd34.center} + 12.5
```

The graphical **Set link...** dialog creates these paths automatically, so normal GUI use does not require knowing curve or component identifiers. The canonical expression is still shown as a tooltip and remains available to advanced workflows, Python code, and machine-readable exports. For a link between different spectra, select both spectra and use **Global simultaneous** mode; CurveMole blocks incompatible fit modes with an explicit message.

### 9.4 Link evaluation

Expression links and source-dependent bounds form a directed dependency graph. CurveMole resolves the graph before fitting, detects cycles, and reevaluates the relations at each model evaluation. Equality and advanced-expression targets are derived values rather than independent optimizer variables. If covariance is available, their standard errors are propagated numerically from free parameters. Targets with **At least**, **At most**, or **Similar** relations remain independent optimizer variables constrained by their current allowed intervals.

### 9.5 Safe expression language

Formulas and links are parsed as a restricted expression tree. They are never passed to unrestricted Python `eval` or `exec`.

Allowed arithmetic includes `+`, `-`, `*`, `/`, power `**`, remainder `%`, unary signs, comparisons, boolean combinations, and conditional expressions. Constants are `pi`, `e`, and `inf`.

Allowed functions are:

```
abs, sqrt, exp, log, log10,
sin, cos, tan, arcsin, arccos, arctan,
sinh, cosh, tanh,
erf, erfc,
minimum, maximum, clip, where, heaviside
```

Attribute access, imports, indexing, comprehensions, arbitrary function calls, file access, and network access are not part of the expression language.

# 10. Fitting

## 10.1 Pre-fit checklist

Before pressing **F5**:

1. confirm the active and selected curves; use **Select all** / **Deselect all** when appropriate;
2. inspect invalid and masked points;
3. choose a physically defensible background and peak count;
4. provide reasonable initial values;
5. add justified bounds for poorly determined parameters;
6. inspect links for cycles or unintended dependencies;
7. check that each selected model contains at least one free parameter.

## 10.2 Fit modes

### Single / independent

Each selected curve is fitted as an independent problem. Results are then combined for reporting. Parameter links between different independently fitted curves cannot act as a simultaneous cross-curve constraint; use Global mode for that purpose.

**CPU processes (independent fits)** in the advanced settings runs multiple spectra in separate processes, up to the selected count and the available CPUs. Both built-in functions and user-defined formulas support this, including mixed models. Formula definitions are reconstructed in each worker; fitted parameters, covariance and completed-spectrum results are returned to the main workspace. Executable plugin functions and plugin solvers retain their single-process execution path.

### Sequential

Sequential mode is a propagating refinement. Choose an **Initial source spectrum** whose model and parameters have already been inspected and approved. The source is not refitted. Its model is cloned to the next selected spectrum, fitted there, and the result becomes the source for the following spectrum. Selected spectra before the chosen source are excluded; target spectra do not need models in advance.

Use **Functions excluded from copying** to select any source functions to leave out. Functions marked as background are excluded by default; uncheck one to copy it. Functions already present on each target retain their values, metadata, constraints, and order during copying; the fit still respects their fixed/free state. A function with the same internal identity is kept rather than duplicated. Distinct functions remain distinct even if their names match. Target-local functions are not carried into later spectra. The selection is also preserved when resuming a paused sequence. This is independent of ignoring functions in the parameter-change pause monitor.

For newly copied functions, parameter values and structure are propagated. The dialog separately controls whether to preserve bounds, fixed/free state, internal links, background tags, enabled/disabled state, and composition/grouping. References inserted with **Same spectrum as this parameter** are remapped to corresponding target parameters; explicitly selected fixed spectra keep their original references. Cross-spectrum links and source-dependent bounds require Global simultaneous fitting rather than independent or

sequential fitting. Structural metadata required by the function, such as spline nodes and custom-formula metadata, is always retained.

Two optional safeguards can pause a sequence:

- normalized residual RMSE worsens by both a selected factor and a minimum fraction of the signal scale; defaults are 2.5 times and 2 percent;
- a free parameter changes by more than a selected normalized amount; the default is 75 percent.

Specific source functions can be excluded from the parameter-change trigger while remaining fully copied, fitted, and included in the residual. If fitting or a safeguard pauses the sequence, the affected spectrum becomes active and a persistent **Continue sequential fit** button appears in the status bar. Inspect or edit that spectrum, then continue; the accepted current model becomes the new propagation source. CurveMole does not silently skip a paused spectrum. The warning explains the affected spectrum, reason, measured change and threshold, and next action. **Last pause reason**, beside the sequential controls, reopens the latest warning while the sequence is paused, including after a manual fit used to correct the paused spectrum. The button disappears when the sequence completes or is terminated. The message is saved with the project and is replaced when a later sequential fit pauses.

#### Global simultaneous

All selected residual arrays are concatenated into one optimization problem. Models may differ between curves. Parameter links can connect curves and express shared, offset, scaled, or otherwise related values. Source-dependent **At least**, **At most**, and **Similar** bounds can also connect spectra; include every referenced spectrum in the global fit.

### 10.3 Per-spectrum weights

The Fit dialog uses the same **Active series / All series** scope for independent, sequential and global fits, with colored series headings. Hidden spectra are excluded from the plan; the sequential source chooser follows the same scope. Weights and checkbox choices are retained while switching scope. Function selection and enabled/background checkboxes are described in Section 5.4.

The Fit dialog accepts a positive numeric spectrum weight for every selected curve. If $s_j$ is the spectrum weight, all point residuals from spectrum $j$ are multiplied by $\sqrt{s_j}$.

**Scale each spectrum to equal numerical contribution** additionally divides the residual vector of each curve by the square root of its number of usable points. This prevents a densely sampled curve from dominating only because it contains more rows. It does not normalize signal amplitude or uncertainty and is never enabled silently.

### 10.4 Local least squares

The default is local nonlinear least squares using SciPy. CurveMole automatically selects:

- Levenberg-Marquardt for an unbounded ordinary least-squares problem;
- trust-region reflective when bounds or a robust loss require it.

The graphical dialog exposes the maximum number of evaluations and confidence level. New fit plans default to 1,000 evaluations and use a step tolerance that permits a successful early stop when parameter changes become numerically negligible. During local optimization, the status progress is updated and the plotted trial model is refreshed every 20 evaluations. These refreshes preserve the user's current zoom and do not replace the stored parameters unless the fit completes successfully. Advanced algorithm options, the Python API and YAML settings expose tolerances, scaling, local method, random seed and Differential Evolution controls. The solver menu also offers explicit TRF, Dogbox, and LM, plus Nelder-Mead, Powell, and L-BFGS-B. The latter three minimize the selected loss directly; they can be useful for difficult models but may take more evaluations. LM needs unbounded parameters and `linear` loss. Hover over a solver to see its intended use.

### 10.4.1 Advanced algorithm options and Reset default

The default remains **Local least squares (automatic)** with **linear** loss. **Advanced algorithm options** shows only the controls relevant to the chosen solver. It also contains **Use data errors / weights in the fit**, enabled by default; this choice applies to independent, sequential and global fitting and replica refits. Numeric fields accept scientific notation such as `1e-8`; tooltips explain each option.

| Algorithm | Configurable controls |
|---|---|
| Automatic, TRF, Dogbox, LM | `ftol`, `xtol`, `gtol`, `x_scale`; automatic mode also offers the local method |
| Differential Evolution | Generations, population multiplier, mutation minimum/maximum, recombination, relative/absolute tolerance, seed and automatic search limits; least-squares controls apply to local refinement |
| Nelder-Mead | Iteration limit, `xatol`, `fatol`, adaptive coefficients and optional initial simplex |
| Powell | Iteration limit, `xtol`, `ftol` |
| L-BFGS-B | Iteration limit, `ftol`, projected-gradient tolerance, line-search steps and correction history |

Maximum evaluations remains configurable for all built-in solvers. For Differential Evolution it limits the local refinement; generations/population control the global search. Numerical Jacobian probes and L-BFGS-B numerical gradients may require additional model calls beyond the solver's reported evaluation budget.

An iteration limit of `0` retains the minimizer's automatic limit. An optional Nelder-Mead simplex is a JSON matrix with N+1 rows and N columns in the fitter's free-parameter order (selected curve, component and parameter order, including referenced link dependencies). Leave it blank for normal automatic initialization. Vertices must satisfy bounds. LM requires unbounded parameters and linear loss.

**Reset default** restores all algorithm/loss parameter fields, evaluation budget and confidence level. It preserves the selected algorithm and loss, fit mode, spectrum selection/weights and model parameters. Defaults include 1,000 evaluations, `ftol=1e-10`, `xtol=1e-8`, `gtol=1e-10`, `x_scale=jac`, and loss scale 1. DE defaults are 400 generations, population multiplier 15, mutation 0.5-1, recombination 0.7, relative tolerance 0.01, absolute tolerance 0 and seed 1729. Nelder-Mead and Powell tolerances default to `1e-4`; L-BFGS-B uses `ftol=2.220446049250313e-9`, `gtol=1e-5`, 20 line-search steps and 10 corrections. These are starting configurations, not guarantees for every dataset.

### 10.5 Robust losses

Available losses are:

- `linear` for ordinary least squares;
- `soft_l1`, which smoothly weakens the effect of large residuals;
- `huber`, squared near zero and nearly linear for large residuals;
- `cauchy`, which strongly suppresses very large residuals.

Hover over the loss options in the Fit dialog for short descriptions. Losses alter the penalty assigned to residuals, not the mathematical peak shape.

Robust loss can reduce the influence of large residuals, but it is not a substitute for understanding outliers, detector artifacts, or an incomplete model. AIC, AICc, and BIC are reported only for the linear loss. Robust-loss covariance uses a sandwich estimate, and CurveMole recommends explicit resampling.

For `soft_l1`, `huber` and `cauchy`, the **Loss scale (f_scale)** field appears. It must be positive and sets the transition between small and large residuals. It is hidden for `linear`, where it has no effect. The scale is expressed in the residual units passed to the optimizer: signal units without uncertainty weighting, standardized residual units with `sigma_y`, further modified by spectrum weights and equal-contribution scaling. The default of 1 is not an automatic noise estimate; choose a scale appropriate to the data. Reset default restores it to 1.

### 10.6 Differential Evolution initial search

Choose **Differential Evolution + local refinement** when a local initial estimate is not sufficient. Differential Evolution searches globally, then passes its best point to the local least-squares solver.

When a free parameter lacks a finite bound, CurveMole creates one around its current value. By default, the search extends 50% of the absolute current value below and 50% above it. At zero, a scale based on the plotted data is used. Edit the separate **Below** and **Above** percentages in the Fit dialog; existing explicit bounds always take precedence. These temporary search limits do not change the parameter table or the saved model. Inspect them conceptually before interpreting the result: a global search cannot discover a solution outside its search interval. It is usually much slower than a well-initialized local fit.

### 10.7 Cancellation and failed fits

Use **Fit > Cancel running task** to request cancellation. The request is checked during residual evaluation and resampling. If a built-in fit or uncertainty task does not respond within five seconds, CurveMole retires it and releases the workspace. Calculations own detached inputs; late results from a retired task are ignored. Previous valid results remain available. An uncooperative executable plugin may continue computing in its detached daemon thread, but cannot keep the project locked or prevent the application from closing. Save and autosave can capture the current project while a calculation is active. Finalization keeps progress below 100% until the task has actually finished.

If the solver raises an error or does not converge, CurveMole restores the last valid parameter values instead of committing an incomplete optimizer vector. The curve is marked Failed, the error is written to the Log, and the last successful result remains separate from the failed attempt.

### 10.8 Common causes of failure

- no free parameters;
- fewer than two usable points;
- initial value outside bounds;
- non-finite model values;
- division by zero in a composed model;
- a missing or cyclic link;
- a linked value outside its bounds;
- incompatible static and source-dependent bounds, or an infeasible dynamic relation;
- an invalid or overly narrow Differential Evolution search interval;
- an underdetermined model;
- inadequate initial values or maximum evaluations.

## 11. Results, statistics, and diagnostics

### 11.1 Visual results

The main plot shows data, total model, individual component curves, and residuals. The parameter table shows fitted values and standard errors. A curve state changes to Fitted only after solver success.

The full numeric result is stored in the project and written to exports. Preview 0.42.0 does not yet provide a single comprehensive on-screen results table, so use the analysis bundle for archival inspection.

### 11.2 Reported global statistics

Let $N$ be the number of usable observations and $k$ the number of independent free parameters. CurveMole reports:

| Statistic | Definition or interpretation |
|---|---|
| Degrees of freedom | $N - k$ |
| RSS | $\sum_i (y_i - f_i)^2$ without point weighting |
| RMSE | $\sqrt{\mathrm{RSS}/N}$ |
| Chi-square | Sum of squared weighted residuals |
| Reduced chi-square | Chi-square divided by $N - k$ when positive |
| Descriptive $R^2$ | $1 - \mathrm{RSS}/\mathrm{TSS}$ |
| AIC | Gaussian ordinary least-squares expression when applicable |
| AICc | Small-sample correction when $N > k + 1$ |
| BIC | Gaussian ordinary least-squares expression when applicable |

The descriptive $R^2$ can be misleading for baselines, constrained fits, or models without an intercept. It is included as a familiar descriptor, not a universal model quality score.

### 11.3 Per-curve statistics

Each curve output records the original usable indices, x, observed y, fitted y, residual, weighted residual, N, RSS, RMSE, and descriptive $R^2$. The global result combines all curves in the plan.

### 11.4 Covariance and correlation

For ordinary least squares, covariance is derived from the fit Jacobian. If all curves provide `sigma_y` or declared confidence-interval Y errors and weighting is enabled, the uncertainty is treated as absolute by default and the covariance is not rescaled by residual variance. Otherwise residual variance scales the covariance.

CurveMole uses a pseudo-inverse when the Jacobian is rank deficient and reports a warning. Covariance is unavailable when degrees of freedom are not positive. A warning is also generated when at least one free-parameter correlation has absolute value 0.95 or greater.

Symmetric normal intervals respect static bounds and any global limits implied by the relations. For **At least**, **At most**, or **Similar** with a free source, the source and target can move together: the target's marginal confidence interval is not clipped to a neighborhood that freezes the source at its fitted value. Reported effective lower/upper limits describe the allowed range at the optimum; confidence intervals use the applicable static/global limits instead of these conditional limits. A parameter at a bound is flagged because its true uncertainty is generally asymmetric and not adequately summarized by a symmetric covariance interval.

### 11.5 Residual diagnostics

Open **View > Diagnostics** after a fit. The summary reports:

- Durbin-Watson statistic;
- number of residuals with absolute standardized value at least 3;
- lag-1 autocorrelation;
- warnings for potential outliers and autocorrelation beyond an approximate 95 percent white-noise band.

The analysis bundle exports the autocorrelation sequence for every fitted curve. A structured residual pattern usually indicates that the model, background, weighting, or data treatment deserves review.

## 12. Explicit uncertainty analyses

Open **Fit > Uncertainty Analysis** only after a successful fit. These calculations run in the background, can be cancelled, and may be substantially slower than the baseline fit. **Fit covariance** displays the uncertainty already calculated by the fit without running replicas. Monte Carlo, residual bootstrap and block bootstrap default to **Adaptive - stop when intervals stabilize**. **Fixed count** remains available with an initial count of **200 replicates** (not optimizer iterations). Selecting it shows a notice: fixed count does not check interval stability. Prefer Adaptive; use fixed count for tests or when a quick analysis is needed. The optimizer's default evaluation budget is unchanged.

**Replica timeout (seconds)** limits each Monte Carlo or bootstrap replica to **60 seconds** by default and can be changed before running an analysis. The timer starts when that replica begins calculating, excluding time waiting in the queue, and resets for every replica. It applies in fixed and adaptive modes, with one or multiple CPU processes. Built-in functions and custom formulas run in isolated workers, allowing CurveMole to stop even an unresponsive replica. Executable plugin functions and custom fitter subclasses use cooperative cancellation for the replica deadline; the task cancellation safeguard described in section 10.7 also applies to these analyses.

If a replica exceeds the limit, the entire active analysis stops and is marked **Uncertainty analysis failed (timeout)** for the selected method, with an amber status and a warning icon beside the spectrum. The results panel also shows an explicit timeout warning. The existing fit and previous completed analyses remain available; spectra already completed in the batch keep their results. A later successful analysis clears the failure tag. The timeout is recorded under **Technical details** and saved with results.

Ordinary failed replicas produce one warning for the spectrum, showing failed attempts and successful replicas used for the intervals. Even a single failure is reported, since excluding unsuccessful fits may bias intervals. This warning does not mark every parameter as **Attention**: parameter assessments retain their own diagnostics and may remain **OK**, meaning that those checks flagged no local issue. The spectrum warning still applies to the analysis as a whole. If the adaptive limit is also reached, both warnings are shown. Too few successful samples and parameter-specific diagnostic issues continue to be reported in assessments.

For Monte Carlo and both bootstrap methods, **Replicate mode** offers **Fixed count** or **Adaptive - stop when intervals stabilize**. Adaptive mode defaults to:

- 500 successful replicates for the first interval estimate;
- 200 additional successful replicates per checkpoint;
- less than 5% endpoint change, measured against each interval's current width;
- three consecutive stable checks;
- a hard limit of 10,000 attempts, including failed fits.

All five settings are editable. Each checkpoint recomputes percentile intervals using all successful replicates accumulated so far. Both endpoints of every free parameter must satisfy the tolerance; an unstable check resets the consecutive counter. With these defaults the earliest stable stop is at 1,100 successful replicates. Failed fits do not count toward the initial sample or subsequent blocks, but do count toward the maximum attempts. If the limit interrupts a block, the final intervals still use all successful replicates, and the result explicitly reports that stability was not reached. Identical zero-width intervals are treated as numerically stable; changing to a zero-width interval is not a stable check.

The limit applies separately to each independent spectrum, or once to a joint global fit. The report records the actual successes, failures, attempts, stopping reason, settings and checkpoint history, including when the project is saved and reopened. Reaching the limit

before stability is flagged in the assessment, in a prominent results warning, and beside the spectrum with an amber status and warning icon. The intervals are retained, but the required stability has not been demonstrated. Warnings follow the method selected in the analysis panel and survive save/reopen; switching to another method shows that method's status. This is a numerical stability criterion, not a guarantee of statistical accuracy or model validity. Covariance and profile likelihood do not use adaptive replicate counts.

**Delete selected analysis from all spectra** removes the method currently chosen in this panel's **Method** field from the entire project. It removes its reports, saved samples and timeout warnings, updates the spectrum tags and available display methods, and preserves all other methods, spectra and fitted parameters. **Undo** and **Redo** restore or remove it. The button is disabled in read-only projects and while a background task is running. The panel scrolls when its controls and results exceed the available height. The deletion applies to the shared selected method, independently of **Run on**. Both the analysis panel and Model and parameters show this same method. Save the project to persist the removal. Previously saved project files are not rewritten until saved; other archives and exported files are not deleted by this button.

**CPU processes** sets the maximum number of independent spectra analyzed in parallel, with remaining spectra queued. The default is up to four processes, limited by the available CPU count. Built-in functions and custom formulas support this parallelism for both selected spectra and all fitted spectra, in fixed and adaptive modes. A single spectrum or joint global analysis can parallelize its replicates instead; profile scans can parallelize their grid points. Analyses involving executable plugin functions use one process. Raising the process count can increase memory use, and it does not change the requested replicate count. The actual worker process count is recorded under **Technical details**.

**Run on** selects the active spectrum, spectra selected in the project tree, or all fitted spectra. Each independent spectrum uses its own last successful fit, even if another spectrum was fitted more recently. Switching the active spectrum updates the report shown in the panel. Results are saved separately for each spectrum and method. The spectrum list's **Uncertainty analysed** tag refers to the method currently chosen in **Method**. Switching from bootstrap to Monte Carlo removes the completed tag from spectra without a saved Monte Carlo analysis; switching back restores the bootstrap tag. A saved analysis of the selected method is marked outdated after data/model changes. Hover over the state to see the selected method and all saved analyses. Changing the method preserves the selection and does not rerun or delete any analysis. In **Model and parameters**, click **Analysis - / +** in the parameter-table header to open the **Displayed uncertainty** menu. Its checked entry selects which saved analysis supplies the coloured **Analysis − / +** column for every spectrum. The choice is universal within the project: switching spectra or using **Show all functions** does not change the selected method. The selector lists methods recorded for at least one spectrum in the project. A spectrum without the selected analysis shows a dash in the analysis column; it does not fall back to another method. The sole available method is displayed automatically, including after reopening older projects without a saved display preference. If several analyses are available, the selector remembers one project-wide choice after saving and reopening. Finishing a background analysis preserves a different method selected while it was running. Older per-spectrum preferences migrate to the first available saved choice in spectrum order. Fit covariance analyses use their recorded confidence limits. When no analysis is available anywhere in the project, clicking the column heading explains that an analysis must be run first and offers **Open Uncertainty Analysis**, which opens that panel. The menu is attached directly to the error-column heading; the uncertainty panel's **Method** control chooses the same method to show, inspect or run. Changing either control updates the other and all spectrum tags. Selection does not run an analysis: use **Run explicit uncertainty analysis** to calculate it. If the chosen method has no recorded result, errors remain blank. The header menu marks it as **no recorded result** when other saved methods are available; it does not silently substitute another method. Available saved methods can still be selected. With **Show all functions**, the empty-state link activates the spectrum owning the displayed function before opening its analysis panel. The project-wide selector remains available when several functions are selected and parameter rows are hidden. The adjacent **±1σ** column always retains the

original fit error. The analysis column shows the distances below and above the fitted value separately. It shows a dash when a parameter has no valid interval or the interval excludes the original fitted value; inspect the assessment for details. The parameter CSV retains `parameter_err` for the fit error and adds `parameter_analysis_err_minus`, `parameter_analysis_err_plus`, and the selected method when applicable. Saved analyses remain selectable after reopening, including reports from older project formats and projects opened read-only. When data/model changes make an analysis outdated, its intervals are shown in amber with **Recorded:**. They refer to the earlier fitted value, given in the tooltip, and are not uncertainties of the current parameter values. Refit and rerun the analysis to obtain current intervals. Current-parameter CSV errors exclude outdated analyses; the uncertainty report retains the historical results and their assessment. In **File > Export functions parameters**, **Also export covariance matrices** is off by default. When selected, a sibling `*_covariance` folder contains an `index.csv` and labelled matrices: the fit covariance and, separately, sample covariance for the selected bootstrap or Monte Carlo analysis when raw samples are available. Global fits retain cross-spectrum terms in one joint matrix. Profile likelihood provides intervals, not a sample covariance matrix. Running an uncertainty analysis never changes the original fit covariance. Older projects may need a new fit for spectra whose individual baselines were not recorded. Profile likelihood accepts one active spectrum at a time; coupled global fits are analyzed together when resampling is requested.

All methods display a table with spectrum/function names, parameter, original fitted value, confidence interval endpoints and assessment. Click the assessment to read its explicit reasons, including the names and correlation coefficients of strongly correlated parameters. Intervals are not symmetric +/- errors and their midpoint is not necessarily the original fitted value. Technical paths, seed and the baseline timestamp remain available under **Technical details**. The confidence level is configurable for resampling/profile; covariance shows the recorded fit's confidence level. Changing the method recalls its most recent recorded analysis.

| Assessment | Meaning |
|---|---|
| OK | No diagnostic issue is flagged by these checks; this does not establish practical adequacy |
| Attention | Bound contact, strong correlation, unstable interval endpoints, an open profile scan, outdated results, or another stated diagnostic caveat |
| Critical | No usable interval, too few successful replicas, rank-deficient covariance, or an interval spanning most of the allowed range |
| Fixed | Parameter was fixed; its uncertainty was not estimated |

These are deterministic diagnostic heuristics, not universal scientific thresholds: strong correlation means absolute r >=0.95; coverage of at least 80% of a finite allowed parameter range is flagged; fewer than 20 successful replicas is critical, and fewer than 200 is flagged for unstable percentile endpoints. Correlations use empirical samples for resampling when available, otherwise the fit covariance. A parameter touching a bound is not necessarily physically wrong. Click the assessment to see why it is flagged; no color certifies the correctness of the physical model. Analyses describe their recorded baseline; editing data or models requires refitting before resampling. Reports remain inspectable but are marked outdated when appropriate. Failed-replica counts alone do not change a parameter's assessment. A parameter can therefore be **OK** while its spectrum has a failed-replica warning. Read both the parameter assessment and the spectrum-level warning before interpreting the interval; **OK** does not mean that every replica succeeded or that the analysis has been scientifically validated.

### 12.1 Parametric Monte Carlo

This method generates synthetic y values from the fitted model plus independent normal noise using imported absolute `sigma_y`, or a two-sided Gaussian approximation using the lower/upper Y error half-widths and their declared confidence. The latter transforms negative/positive standard-normal draws with the respective scale; it preserves the declared central interval and has its median at the fitted prediction. Every selected curve must provide valid Y errors on the fitted rows. Noise is still generated from these errors when weighting is turned off in the Fit dialog. The chosen approximation and input confidence are recorded in the analysis settings.

Each synthetic data set is refitted using the local solver. The result records the requested, completed, and failed replicates, seed, free parameter paths, samples, empirical intervals, and failure messages.

### 12.2 Residual bootstrap

Residuals from fitted rows are centered, sampled independently with replacement, added to the fitted model, and refitted. With Y errors, residuals are first standardized using each observation's lower/upper scale, then reconstructed with each destination row's appropriate scale. This accommodates the supplied relative noise levels without adding measurement noise a second time. Masked rows do not supply residuals. This assumes exchangeability of the standardized residuals and can be inappropriate when residuals remain serially correlated or heteroscedastic.

### 12.3 Block bootstrap

The block bootstrap resamples contiguous circular residual blocks. With Y errors it resamples standardized residuals and restores the destination row's noise scale, as in residual bootstrap. Enter a block length or choose Automatic. The automatic estimate uses the first lag at which the absolute residual autocorrelation falls below $e^{-1}$, subject to practical limits.

Block resampling is often preferable for spectra or kinetic traces with local correlation, but the block length remains a scientific modeling decision.

### 12.4 Profile likelihood

Choose one free parameter that is not fixed or equality-linked. CurveMole fixes it across a finite grid, refits all remaining free parameters, and compares chi-square with the baseline. The default grid spans approximately three covariance standard errors on either side, or a fallback span when no standard error is available, while respecting bounds.

The default uses 31 grid points and a one-parameter chi-square threshold. The grid size, confidence level and lower/upper scan limits can be changed in the panel; blank limits select the automatic range, and explicit limits must respect parameter bounds. Failed grid points are counted. The desktop profile currently requires a single-spectrum baseline; it does not profile a global multi-spectrum fit. A profile interval is more informative than a symmetric standard error near bounds or in nonlinear problems, but grid resolution should be considered when interpreting endpoints.

### 12.5 Reproducibility and failed replicates

The default seed is 1729 unless changed through programmatic settings. Resampling outputs record configuration and up to the first 100 failure messages. Do not report an empirical interval without also reporting how many replicates completed and failed. Selecting **Uncertainty intervals, assessments and matrices** in the export dialog also writes `uncertainty/parameter_assessments.csv` for the recorded analyses, including spectrum/function names, parameter paths, fitted values, interval endpoints, confidence level, method and reasons. Covariance/correlation matrices remain available in the same export option. This CSV contains parameter assessments, not the spectrum-wide failed-replica counts. Keep the `.fitproj` project, or select a project copy in the analysis bundle, to retain the recorded counts, failure messages and adaptive configuration alongside the parameter assessments.

# 13. Projects, saving, and recovery

## 13.1 The `.fitproj` format

A `.fitproj` is a ZIP container with:

- a versioned JSON manifest;
- original arrays stored as NumPy `.npy` payloads;
- uncertainty and weight arrays when present;
- mask arrays;
- transformation operands;
- models, links, custom functions, results, history, and UI/export state;
- SHA-256 checksums for every binary payload.

NumPy arrays are loaded with `allow_pickle=False`. CurveMole projects never require executing a Python pickle. The format is documented in project-format.md.

## 13.2 Atomic saves

CurveMole writes a temporary sibling file, validates the archive, and then atomically replaces the destination. A failed save should therefore not partially overwrite a previous valid project.

## 13.3 Project locks and read-only mode

Opening a project creates a sibling `.lock` file containing process and host information. If another instance already holds the lock, CurveMole opens the project read-only. Use **Save project as** to create an editable copy. A stale lock left by an abnormal termination may need manual inspection and removal after confirming that no other instance is using the project.

## 13.4 Normal save and portable copy

**Save project** updates the current project path. **Save portable copy** writes a complete separate `.fitproj` without changing the active project path. Since project data are already embedded, the portable copy is intended for sharing or archiving a snapshot.

Projects are not encrypted. Use suitable filesystem permissions or encrypted storage for confidential data.

## 13.5 Autosave recovery

Manual Save, Save As and portable-copy writing also run on a background worker. The status bar identifies the file being saved while the plot and controls remain available. Edits made during writing remain unsaved; a completed snapshot does not clear recovery copies for newer changes. Close/Open waits for the save outcome with a responsive interface and asks again if newer unsaved changes remain.

Autosave runs in a dedicated background thread, so compression, disk writing and integrity checks keep the plot and controls available. Each backup represents a coherent snapshot from the start of the operation; edits made while it is being written remain unsaved and are included in a later backup. Only one autosave runs at a time, with overlapping requests combined into the latest requested state. Built-in fits and uncertainty analyses work on detached inputs, so manual saves and autosaves include the current project and results already committed without waiting for the entire calculation. Later completed results become new unsaved changes. Undeclared external background tasks still defer snapshotting until their worker has finished.

Every ten minutes, a modified revision is written to the operating system's CurveMole user cache. An unchanged revision does not create another recovery. The six newest valid, distinct recovery files for a project are retained.

CurveMole offers **Recoverable sessions** at startup for unresolved sessions, including crashes and exits without a successful Save or explicit Discard. The same list is available from **File > Recoverable sessions**. Sessions are grouped by project name with their recovery date and copy count. **Recover** opens the newest valid copy by default; an older retained copy can also be selected. The recovered workspace is unsaved and does not overwrite the original project.

The proposal explicitly names the original project/file and the recovery file it will open. Each project lists named **backup 1** through **backup 6** entries, with dates and the newest valid copy selected. Older copies without stored names use the internal project name or an identifier to keep unnamed sessions distinct.

**Decide later** retains the copies and the proposal for the next launch. **Delete selected copies** removes individual copies or selected project groups after confirmation. Saving all current changes successfully or explicitly choosing **Discard** when closing or switching projects removes its recovery copies. Cancelling a dialog or a failed save preserves them. Other projects' copies remain.

The application counts completed recovery files once per minute. Startup recommends periodic manual cleanup to limit disk usage and opens the recovery manager when the total reaches a new multiple of 50: 50, 100, 150, 200 and so on. Each warning is shown once per reached band, remembered across restarts. If several thresholds are crossed between launches, only one warning is shown for the current count. Cleanup to fewer than 50 copies restarts the warning cycle. Copies from multiple projects can be kept indefinitely within each project's six-valid-copy rotation; there is no automatic age expiry. Damaged completed copies can be deleted manually. Abandoned staging files are automatically removed only when their owning process is known to have exited; files still in use are retained.

**File > Recent projects** lists up to ten successfully opened or saved projects; choose an entry to reopen it, or clear the list from that menu.

### 13.6 Unsaved changes

An asterisk in the window title marks a dirty project. New Project, Open Project, and Quit ask whether to save, discard, or cancel when unsaved changes exist.

## 14. Exporting an analysis bundle

### 14.1 Starting an export

Choose **File > Export analysis bundle** or press **Ctrl+E**. Export mode and export content are independent.

The existing save modes remain:

- leave both mode checkboxes clear to create a new export in the selected directory;
- **Create versioned export** to create a timestamped subdirectory;
- **Update existing CurveMole-owned files after confirmation** to update files that CurveMole can verify as belonging to the current project export.

Under **What to export?**, select the desired outputs. Only **Fit results** is checked by default.

### 14.2 Default fit-results file

The default export creates only `fit_results.csv`. No additional visible file and no empty subdirectory is created.

Each row represents one parameter of one model component and includes the series, curve and curve ID, component and component ID, displayed function and function ID, enabled/background state, composition operator, parameter name and full parameter path, fitted value, standard error, confidence interval, bounds, fixed/link state, unit, human-readable value with uncertainty, and derived area/FWHM where defined.

This format makes the association between every data curve, the functions used to fit it, and the fitted parameters explicit in a single table.

### 14.3 Optional outputs

The dialog can additionally export:

- data plus fitted-function wide CSV tables;
- a tidy CSV table for Python;
- machine-readable fit results as JSON;
- a reusable `.fitmodel`;
- a `.fitproj` project copy, optionally including full uncertainty samples;
- the main plot as PNG and/or SVG;
- compact HTML and PDF summaries;
- a full reproducibility HTML report;
- the laboratory notebook as UTF-8 text;
- all recorded settings for the last fit as CSV;
- covariance and correlation matrices when available;
- residual autocorrelation diagnostics when available;
- an export README.

Directories such as `data/`, `python/`, `figures/`, `report/`, `uncertainty/`, and `diagnostics/` are created only when a selected output actually writes a file there. An option whose required fit result is unavailable does not create an empty folder.

### 14.4 Safe overwrite policy

New exports store their ownership list in the project export configuration rather than creating a hidden sidecar file. This allows the default export to contain truly only `fit_results.csv`. Existing `.curvemole-export.json` manifests from older versions are still read for backwards-compatible updates.

When **Update existing** is selected, CurveMole refuses to overwrite a colliding file unless it can verify that the file belongs to the current project export. Unrelated files in the directory are preserved.

### 14.5 Wide data tables

Optional wide CSV files are intended for direct inspection and tools such as Origin or QtiPlot. Depending on data and model, columns include:

- x and y;
- total fit;
- residual as data minus fit;
- total background;
- every enabled component;
- `sigma_y` or point weight;
- mask flag.

Rows remain aligned with the complete curve, including masked and invalid entries.

#### 14.5.1 Exporting optimizer and loss settings

**Last fit: all solver/loss settings (CSV)** writes `fit_settings.csv`, containing all recorded `FitSettings` fields, including advanced solver parameters, loss and `f_scale`. Each row includes the fit timestamp, mode and curve IDs. It describes the selected last fit result, not unsaved edits in the Fit dialog; fields for inactive algorithms are retained for completeness. No settings file is produced without a fit result. `fit_results.csv` separately contains all model parameter values, bounds, links and available errors. The JSON result includes the same optimizer settings; the full reproducibility HTML report also shows them as a table.

### 14.6 Tidy data and JSON

`python/data_tidy.csv` stores one quantity per row with series, curve, component, x, value, mask, and validity fields. `python/results.json` records schema version, application version, result metadata, parameters, statistics, warnings, and settings without duplicating large numeric arrays.

### 14.7 Reports and reusable files

- `summary_report.html` is a compact human-readable report.
- `report/full_reproducibility.html` includes project metadata.
- `report/summary.pdf` contains a summary plot and statistics.
- `.fitmodel` stores the first project model and custom formulas for programmatic reuse.
- `.fitproj` is a complete portable project snapshot.

The exported report PDF is an analysis summary, not the CurveMole software manual.

### 14.8 Exporting one numeric file per spectrum

Choose **File > Export spectra and fit curves...** or press **Ctrl+Shift+E** when the result should be immediately replottable as one multi-column table per spectrum. This is separate from the structured analysis bundle. Choose any project spectra and any of these options:

- export the measured spectrum with its fitted background subtracted;
- omit masked rows;
- include every enabled individual model function;
- include the combined fitted background;
- include the total fit;
- include residuals as data minus total fit.

The x and spectrum columns are always written. Output names follow `<spectrum>_curvemole.<original extension>`; duplicate names receive a numeric suffix. The original delimiter is retained when practical, otherwise the output is tab-delimited. Headers describe axis labels, units, functions, background state, total fit, and residuals, and never contain whitespace so they remain safe for whitespace parsers. An unknown original extension is preserved.

### 14.9 Exporting function parameters

**File > Export functions parameters...** writes a wide CSV table with one row per project spectrum and two header rows identifying functions and parameters. Each parameter has value and fit-error columns; functions with the same displayed name align across spectra, with duplicate names disambiguated. Spectra that are not currently Fitted retain their row but have blank numeric cells. Saved selected-analysis errors are added as separate minus/plus columns, together with the method. This is a focused numeric export; use the analysis bundle for the full constraints and reproducibility record. **Also export covariance matrices** optionally writes the labelled sibling matrices described in Section 12.

# 15. Command-line interface

## 15.1 General form

```
curvemole [--version] [--json] [--verbose] COMMAND [OPTIONS]
```

Use `--json` for machine-readable output. Normal data go to standard output; logs and errors go to standard error.

## 15.2 Open the GUI

```
curvemole gui
```

The desktop executable can also receive a project or data path as its first argument.

## 15.3 List available functions

```
curvemole functions list
curvemole functions list --json
```

## 15.4 Fit one file

```
curvemole fit spectrum.csv \
  --x wavenumber \
  --y absorbance \
  --function gaussian \
  --background linear \
  --output spectrum.fitproj
```

`--x` and `--y` accept a column name or zero-based index. Optional initial values are `--center`, `--width`, and `--area`. The meaning of `--width` follows the selected function: Gaussian and Voigt use `sigma`, Lorentzian uses `gamma`, and pseudo-Voigt uses `fwhm`.

If no initial peak override is supplied, the CLI uses the strongest automatic positive peak suggestion when available. Background choices are none, constant, or linear.

## 15.5 Fit a series

```
curvemole fit-series scan_01.csv scan_02.csv scan_03.csv \
  --x 0 --y 1 \
  --function pseudo_voigt \
  --background constant \
  --mode sequential \
  --output series.fitproj
```

Modes are `independent`, `sequential`, and `global`. Use `--global-search` to request Differential Evolution. Missing finite bounds are generated from current parameter values; use a YAML workflow or the Python API to customize percentages or explicit parameter bounds.

### 15.6 Inspect and validate

```
curvemole inspect analysis.fitproj --json
curvemole inspect raw_data.dat --json
curvemole validate analysis.fitproj
curvemole validate workflow.yml
```

Inspecting a data file reports detected parsing and a preview. Validating a project checks archive structure and checksums. Validating YAML checks the supported workflow schema and required top-level structures.

### 15.7 Export an existing project

```
curvemole export analysis.fitproj exported_analysis
```

Use `--versioned` for a timestamped subdirectory. Use `--force` only when updating a directory whose CurveMole ownership manifest permits the overwrite.

### 15.8 Exit codes

| Code | Meaning |
|---|---|
| 0 | Success |
| 2 | Command usage error |
| 3 | Data validation error |
| 4 | Fit error |
| 5 | Project format error |
| 6 | Plugin or other CurveMole domain error |
| 10 | Unexpected failure |

## 16. Reproducible YAML workflows

### 16.1 Running a workflow

```
curvemole run examples/gaussian_workflow.yml --json
```

Relative paths are resolved from the workflow file's directory. Schema version 1 supports imports, custom functions, explicitly trusted plugins, models, constraints, fit settings, project output, and bundle export.

#### Run an existing automation from the desktop

The separate **Automations** toolbar at the bottom contains **Run automation**. On the first click, choose a `.yml` or `.yaml` workflow; CurveMole validates and runs it. Later clicks run the same file, including after restarting CurveMole. The tooltip shows its path. Use the arrow beside Run to **Choose automation** or **Stop automation**. Cancelling the file picker leaves the previous choice intact; if the remembered file is missing, Run asks for another file.

The workflow executes in a separate process and uses its own imported spectra, models and output settings. It does not operate on the currently open GUI project. Completion reports the processed spectra, fit convergence and output paths. Relative paths resolve against the workflow's folder. To keep results, specify `export` in the workflow. Reruns respect its overwrite/versioning settings. Stopping or failing does not roll back files already written. Wait for completion or stop the automation before closing CurveMole.

Python plugins referenced by a desktop automation must first be reviewed and enabled in **File > Plugin Manager**. Their source fingerprints are checked again at launch; a workflow does not implicitly approve executable plugins. This control runs existing workflows; it is not a visual workflow editor.

### 16.2 Complete example

```yaml
schema_version: 1
name: Gaussian workflow example
series_name: Synthetic spectra

imports:
  - path: gaussian.csv
    config:
      delimiter: ","
      decimal: "."
      header: true
    columns:
      x: x
      y: [y]
      sigma_y: sigma_y
    aliases: [Synthetic Gaussian]

models:
  - curve: Synthetic Gaussian
    components:
```

```yaml
      - function: constant
        name: Background
        parameters:
          offset: 0.0
      - function: gaussian
        name: Main band
        parameters:
          area: 3.0
          center: 0.3
          sigma: 1.0
        constraints:
          sigma:
            minimum: 0.001
            maximum: 5.0

fit:
  mode: independent
  curves: [Synthetic Gaussian]
  settings:
    solver: local
    loss: linear
    seed: 1729

export:
  project: gaussian_result.fitproj
  bundle: gaussian_export
  versioned: false
  overwrite: false
```

### 16.3 Custom formulas in YAML

```yaml
custom_functions:
  - identifier: exponential_decay
    display_name: Exponential decay
    kind: generic
    formula: offset + amplitude * exp(-x / tau)
    defaults:
      offset: 0.0
      amplitude: 1.0
      tau: 1.0
    bounds:
      tau: [0.000001, 1000000.0]
    derived:
      lifetime: tau
```

Use the custom identifier in a model component exactly like a built-in function.

### 16.4 Trusted plugins

List local plugin manifest paths under `plugins`. Unattended execution also requires one `--trust-plugin IDENTIFIER` argument for each plugin whose Python source may run:

```
curvemole run workflow.yml \
  --trust-plugin org.example.lineshape
```

Trust is never inferred merely because a manifest is present.

## 17. Python API

### 17.1 Stable package-level objects

The package root exports GUI-independent classes including `Curve`, `Series`, `Dataset`, `Component`, `Model`, `Parameter`, `Project`, `FitSettings`, `FitPlan`, `FitResult`, `Fitter`, `FunctionRegistry`, and `default_registry`.

### 17.2 Minimal fit

```
import numpy as np

from curvemole import Component, Curve, FitSettings, Fitter, Model

x = np.linspace(-5.0, 5.0, 1001)
y = 0.1 + 3.0 * np.exp(-0.5 * ((x - 0.4) / 0.8) ** 2)

curve = Curve("Synthetic band", x, y)
model = Model("One Gaussian")
model.add(Component.create("constant", initial={"offset": 0.1}))
model.add(
    Component.create(
        "gaussian",
        initial={"area": 6.0, "center": 0.3, "sigma": 1.0},
    )
)

result = Fitter().fit_single(curve, model, FitSettings(loss="linear"))
print(result.success)
print(result.statistics)
```

The Gaussian expression used to generate y above is height-parameterized, while the CurveMole Gaussian component is area-parameterized. This is why the initial model `area` is not simply the synthetic height.

### 17.3 Bounds and fixed values

```
peak = model.components[1]
peak.parameters["center"].minimum = -1.0
peak.parameters["center"].maximum = 1.0
peak.parameters["sigma"].minimum = 0.05
peak.parameters["sigma"].maximum = 3.0
peak.parameters["area"].fixed = False

for parameter in peak.parameters.values():
    parameter.validate()
```

### 17.4 Cross-curve links

```
source_path = model_a.parameter_path(curve_a.id, peak_a.id, "center")
peak_b.parameters["center"].link = "${" + source_path + "} + 5.0"
```

Construct a Global `FitPlan` containing both curve IDs. CurveMole validates the complete graph before optimization.

### 17.5 Global fit

```
from curvemole.core.fitting import FitMode

plan = FitPlan(
    [curve_a.id, curve_b.id],
    mode=FitMode.GLOBAL,
    settings=FitSettings(loss="linear"),
    spectrum_weights={curve_a.id: 1.0, curve_b.id: 1.0},
    equal_contribution=True,
)

result = Fitter().fit(
    plan,
    {curve_a.id: curve_a, curve_b.id: curve_b},
    {curve_a.id: model_a, curve_b.id: model_b},
)
```

### 17.6 API compatibility during Preview

The package root is intentionally smaller and more stable than internal modules, but version 0.x may still introduce incompatible changes. Pin the exact CurveMole version in automated research workflows and record it with results.

## 18. Function Builder and plugins

### 18.1 Formula Builder

Open **Tools > Function Builder**. Enter:

- an identifier using letters, numbers, and underscores;
- a display name;
- classification as **Peak-shaped function** or **General function**;
- a formula in x;
- optional formulas for derived area and FWHM;
- for peak functions, optional semantic roles for detected parameters.

Background is assigned to components when adding or editing the model; it is not a separate Function Builder classification. **Use manual-point initialization by default** controls the preferred insertion mode for the new function.

CurveMole detects every non-function symbol other than x as a parameter. Newly detected parameters default to 1.0 and are unbounded. After adding the function to the project library, add a component and edit its initial values and bounds in the normal parameter table.

Example:

```
offset + amplitude * exp(-x / tau)
```

Detected parameters are `amplitude`, `offset`, and `tau`.

For a function classified as a peak, the **Peak parameter roles** table can assign at most one of each supported meaning:

- peak position / centre;
- peak height or peak area / integral;
- FWHM, Gaussian sigma, or HWHM/gamma.

These roles tell graphical placement and automatic peak detection how to initialize parameters even when their names are arbitrary. Height and area roles are mutually exclusive. A width role also supplies the standard derived FWHM automatically when a separate FWHM formula was not entered. Leave genuinely unrelated parameters as **Not specified**. An explicitly unassigned role is respected; CurveMole does not guess a meaning from its name.

### 18.2 Custom function persistence

The first time a formula is added, CurveMole asks the user to create a dedicated `my_curvemole_functions` folder at a chosen location or open an existing folder with that exact name. Each definition is saved there as `<identifier>.curvemole-function.json`, registered in the current application, and also embedded in the project. Formula, classification, derived quantities, and peak roles travel together.

CurveMole remembers the selected folder and loads every valid function from it at each startup. Reusable functions therefore appear in the normal Add dialog, Quick Add selector, and automatic peak-shape selector without requiring a particular project to be open. Copy the folder between computers to move a personal function library; open that folder when CurveMole asks for the library location.

Project-contained definitions remain portable. When a project reopens, CurveMole reconstructs and validates each restricted expression. A malformed user function or library file is skipped and recorded in the Log rather than executed. Formula files use the safe expression system and are distinct from executable Python plugins.

### 18.3 Local plugin structure

A local plugin consists of a manifest ending in `.curvemole-plugin.json` and a Python module in the same directory. Example manifest:

```
{
  "identifier": "org.example.my-lineshape",
  "version": "1.0.0",
  "api_compatibility": "1",
  "licence": "GPL-3.0-or-later",
  "capabilities": ["functions"],
  "module": "my_lineshape.py"
}
```

The module exposes `register(api)`. Use `api.register(FunctionDefinition(...))` for model functions or `api.add(kind, id, label, callback)` for exporters, importers, fit algorithms, transformations, analyses, commands, workflows, panels, plot layers and lifecycle notifications. Full contracts and runnable examples are in Creating and managing plugins. Built-ins cannot be replaced through this API; contributed entries carry a plugin-specific symbol and identify the providing plugin in their tooltip.

### 18.4 Trust boundary

Formula Builder expressions are interpreted safely. Plugins are different: a plugin is Python code and can perform any action allowed to the current user account.

CurveMole reads manifests before executing a module and requires an explicit trust decision. Review source, origin, license, and version before trusting. Do not trust a plugin merely because its identifier resembles a known package.

### 18.5 Plugin Manager

Open **File > Plugin Manager**, choose a permanent directory, and scan. Review a candidate, then explicitly trust and load it. Enabled plugins load again on restart, independently of projects. **Disable** unloads contributions; **Remove** also forgets the installation. Source files and project plugin data are retained. Remove any components using a plugin before unloading it. Close the manager to refresh menus.

Changed local source needs approval again. Callback errors disable the plugin; failed registration rolls back additions. After a crash or forced termination, CurveMole disables persisted plugins before importing their code and explains recovery. Normal closure clears the session marker. The integrated validated-plugin browser can download selected plugins through **Install selected**. A **Plugins** status badge checks at startup and hourly for updates to loaded supported plugins, lets you select updates, and indicates when a restart is needed to activate staged code. Open **Plugin updates...** in the manager or click the badge, then choose **Update selected**. **Help > Check for updates** checks both CurveMole and loaded plugins. Checking for an update does not install it automatically. Disabled plugins are not checked; local-only plugins without a catalog entry need manual updates. Manual ZIP download remains an alternative. Updates replace files in the plugin's existing user-selected folder. If that location cannot be replaced, CurveMole asks for another permanent folder; it does not retain plugin code in an internal application folder.

Cancelling leaves the previous installation in place. The saved updated copy remains selected after disabling, restarting and re-enabling the plugin, even if the original folder still contains an older copy. Running plugin code is replaced on the next load. Set `CURVEMOLE_DISABLE_PLUGINS=1` to skip automatic loading manually. This is failure recovery, not a sandbox for Python code.

### 18.6 Creating extensions beyond model functions

Use `api.add("exporters", "csv", "Laboratory CSV", export_csv)` to add an exporter. Its callback receives the chosen destination in `context.path` and a project snapshot in `context.project`. Built-in export commands remain available. `fit_solvers` callbacks receive constrained residuals, Jacobian, initial values, bounds, settings and cancellation. CurveMole computes standard statistics from the returned result without a second built-in optimization.

Importers, transformations, actions and workflows commit snapshot edits as one undoable operation on success. Other callbacks observe snapshots. JSON data in `context.data` is scoped to the plugin and saved with the project. The complete plugin author guide provides manifests, exporter and solver code, all callback contracts, limitations and testing instructions.

For a complete exporter, set the manifest's `module` to `my_exporter.py` and its `capabilities` to `["exporters"]`, then create this module beside the manifest:

```
import numpy as np


def export_csv(context):
    if context.active_curve_id is None:
        return "Select a spectrum first."
    curve = context.project.dataset.curve(context.active_curve_id)
    np.savetxt(
        context.path,
        np.column_stack([curve.x, curve.y]),
        delimiter=",", header="x,y", comments="",
    )
    return f"Exported {curve.name}"


def register(api):
    api.add("exporters", "csv", "Laboratory CSV", export_csv)
```

After loading this plugin, choose **File > Plugins: Exporters > Laboratory CSV** (marked with the plugin symbol). CurveMole asks for a destination and then runs the callback. The integrated export commands stay available. The module uses NumPy, already included with CurveMole; no additional dependencies are required.

### 18.7 Community plugin submissions and validation

To share a plugin, open **File > Plugin manager > Share my plugin on GitHub...**. This opens the submission guide: fork the repository, upload your complete plugin folder at `custom_plugins/by_community/your_plugin/`, then open a pull request to main. For private use, **Choose plugin folder...** installs it on your computer only; loading a plugin does not publish its source. **Browse validated plugins** opens the downloads that have passed the checks.

The repository's custom_plugins folder contains the submission guide and downloadable plugins, including a TSV exporter, Fityk project importer, cosmic-ray removal, x-axis calibration and ruby fluorescence pressure monitor. These are separately installed extensions, not automatically enabled CurveMole features. Authors submit a folder with manifest, module, licence, README and functional tests through a pull request. GitHub checks manifests, actual registered capabilities, loading/removal and tests on Linux, Windows and macOS. Only a successful main run publishes the validated catalog and tested plugin ZIP files. **Browse validated plugins** and **Install selected** provide the normal installation route inside the manager, with an explicit trust decision. Alternatively, use the stable

downloads in the plugin guide or the separate rolling community plugin release, extract the chosen plugin and review/load its manifest in the manager. These direct downloads do not require a GitHub account and are separate from application releases. A source folder or an open PR alone does not indicate validation. Tests demonstrate tested behavior, not complete safety; Python plugins still run with the user's privileges. See the submission guide for required branch protection settings and the limits of validation.

### 18.8 Acquisition processors and interactive plugin panels

Plugins can register `import_processors` for Automatic folder import (Section 6.4). A processor receives only the detached new acquisition and its settings, runs in a background thread, and returns spectra/models/results to be added as one Undo operation. Enabling a plugin does not start acquisition monitoring.

Plugin panels join the shared dock/tab workspace and can be reopened without losing their current widget state. Supported panel services offer project snapshots, persistent plugin settings and control over the owner's import monitor. A panel may open automatically when its plugin is enabled; monitoring still requires an explicit start. Closing a monitor panel does not stop acquisition. Read each plugin's own README for its scientific assumptions, controls and dependencies.

Plugins can also preview corrections, mark candidates and select measured points directly in the main graph, using the spectrum's normal axes, masks and display settings. Previews leave measured data unchanged until the plugin's explicit accept action. Point selection ends with Done/Esc or when another editing tool is activated. Closing or hiding the panel removes its annotations.

Cosmic-ray removal 0.5.0 uses this shared graph for automatic detection, manual detection and candidate review. Select a candidate in its table or click its coloured marker to highlight the affected interval; teal segments show accepted corrections before **Accept modifications**. **Detect manually** adds candidates by clicking measured samples. **Remove manually** opens the native point editor; its drags and deletions take effect immediately with Undo/Redo. This plugin version requires CurveMole 0.42.0 or later.

# 19. Troubleshooting

## 19.1 The application does not start on Linux

Run the AppImage from a terminal to capture the exact error. If Qt reports missing EGL, OpenGL, XKB, or XCB libraries, install the distribution packages listed in Section 3.3. If FUSE mounting fails, use `APPIMAGE_EXTRACT_AND_RUN=1`.

The AppImage is built on Ubuntu 22.04 for broad glibc compatibility. Very old Linux distributions may still require a newer runtime.

## 19.2 Quick Guide, manual, updates, or issue links do not open

CurveMole attempts to launch the host operating system's opener without exposing the AppImage's private Qt and C++ libraries to it. If the opener still fails, the dialog shows the path or URL to copy manually.

Use these direct addresses:

```
https://github.com/SebRoLENS/curvemole
https://github.com/SebRoLENS/curvemole/releases
https://github.com/SebRoLENS/curvemole/issues
```

## 19.3 Windows SmartScreen or macOS Gatekeeper warns

The Preview executables are not yet commercially signed or notarized. Confirm that the file came from the official release, verify its SHA-256 checksum, and use the operating system's documented override only if you accept the risk.

## 19.4 Import produces one column

The selected delimiter is probably wrong. Reopen import and compare whitespace, comma, semicolon, tab, or pipe. A decimal comma file commonly uses semicolon as its delimiter. Confirm the header choice and preview before mapping.

## 19.5 Imported points disappear from fitting

Inspect the Worksheet. A point is excluded when x or y is not finite, `sigma_y` is invalid, a weight is invalid, or the point is masked. Faded markers indicate masks. Use Unmask or undo the transformation that generated missing values.

## 19.6 A peak appears at the wrong height after placement

Built-in peaks use integrated area, not height. The graphical initializer estimates height relative to the existing model and converts it to area. For overlapping peaks or a poor background, adjust the center target, width handle, and area value before fitting.

### 19.7 A spline looks unstable

Use fewer, well-spaced nodes. Place nodes where background information exists rather than at every data feature. Graphically placed node y values are fixed by default; unlock only those that the data can independently determine. Nodes created through the API are ordinary free parameters unless explicitly fixed.

### 19.8 Differential Evolution searches too narrow an interval

Increase the automatic below/above percentages or set physically justified parameter bounds directly. Existing finite bounds override the automatic limits. A linked parameter's resulting value must still satisfy its own bounds.

### 19.9 The fit is underdetermined or covariance is unavailable

Reduce the number of free parameters, fix known values, add justified links, widen the data range, or improve signal information. More optimizer evaluations cannot create information absent from the data.

### 19.10 A parameter remains at a bound

First verify the initial value and physical bound. Then inspect correlations and profile likelihood. A bound-active estimate often has an asymmetric uncertainty and may indicate that the data do not locate the optimum within the allowed region.

### 19.11 A sequential fit pauses

The affected curve becomes active and the status bar displays **Continue sequential fit**. Inspect its propagated model, initial/fitted values, masks, and constraints. The Log and notification identify whether fitting failed, normalized residuals worsened, or a monitored parameter changed beyond its threshold. Correct or approve the model, then continue; this spectrum becomes the new source. For a series with legitimate abrupt changes, adjust or disable the relevant safeguard, or exclude a known discontinuous function from only the parameter-change monitor.

### 19.12 A project opens read-only

Another instance may hold the sibling `.lock` file. Close the other instance, or use Save As. Remove a lock manually only after confirming it is stale.

### 19.13 Export refuses to overwrite

Choose a new directory or a versioned export. If updating an existing bundle, keep the project that records ownership of its exported files and explicitly enable update. Older bundles may instead have a `.curvemole-export.json` ownership manifest; retain it. CurveMole will not claim ownership of unrelated colliding files.

### 19.14 A resampling analysis has many failed replicates

The baseline may be near bounds, poorly initialized, or structurally unstable under perturbation. Increase robustness of the model before increasing replicate count. Report completed and failed counts with any interval.

Even one failed replica produces a spectrum-level warning, not an automatic failure of every parameter. Inspect the cause as well as the count: an isolated difficult fit and repeatedly failing parameter regions can have different consequences. Intervals use successful replicas only. Adaptive checks numerical stability of that successful sample; it cannot demonstrate that excluded failures are harmless. If maximum attempts are reached before stability, the recorded intervals remain available with an explicit warning. A replica timeout instead stops the new analysis and retains any previous completed results. See Section 12 for these distinctions.

### 19.15 An update is detected but cannot be installed automatically

Automatic updating is supported by Linux x86-64 AppImages and Windows x86-64 desktop packages. AppImage and portable replacement requires a writable directory; installed Windows applications use the verified installer. Finish or cancel active tasks. Python/source installations, macOS packages, unsupported architectures, and protected application directories must be updated from the GitHub release page. On Windows, a failed staged update records a helper log in the system temporary directory; include that log when reporting a reproducible updater failure.

### 19.16 Reporting a problem

Use GitHub Issues. Include:

- CurveMole version;
- operating system and package type;
- exact steps to reproduce;
- expected and observed behavior;
- terminal output or traceback;
- a minimal non-confidential data file if possible.

Review every attachment. Diagnostic material must not contain unpublished spectra, personal information, credentials, or proprietary data unless you intentionally make them public.

# 20. Reproducible research, privacy, and citation

## 20.1 Recommended research record

Archive at least:

- exact CurveMole version and DOI;
- original data and import mapping;
- complete `.fitproj`;
- analysis bundle;
- model functions and component order;
- initial values, bounds, fixed values, and links;
- masks and transformations;
- point uncertainties or weights;
- fit mode, spectrum weights, solver, and loss;
- random seed and replicate counts for uncertainty analyses;
- warnings, failed replicates, and residual diagnostics.

### 20.1.1 Laboratory notebook

The **Laboratory notebook** button at the end of the toolbar opens project notes and a searchable description browser as a workspace tab. Use **Add description** from the context menu of a series, individual spectrum, or added fit function to create or edit its description. Function descriptions are specific to the spectrum containing that function, including when sequential propagation preserves a component ID.

Descriptions are grouped by series and spectrum. Names follow renaming and moves. When an object is removed, its description remains in **Deleted items** with the last known name and an explicit deleted status. Restoring the same object with Undo restores its live status. Copying a function does not duplicate its notes.

Edits in the notebook are kept in the current project as you type; save the `.fitproj` file to persist them. Notebook contents are also included in portable project copies. A notebook tab restored at startup displays the current project's contents immediately and follows the project when another file is opened. **Save notebook as text** writes the complete notebook as UTF-8. Alternatively, select **Laboratory notebook (TXT)** in **Export analysis bundle**; this writes `laboratory_notebook.txt` and follows the usual export ownership rules. The notebook can be viewed and exported from a read-only project. Editing is unavailable while a background task runs; a paused sequence allows editing normally.

Series headers are bold and blue, with colors adapted to light and dark themes. A note icon beside a series, spectrum, or fit function opens its description directly. Clearing the description removes its icon and notebook entry.

## 20.2 Local processing and privacy

CurveMole performs fitting locally. It has no telemetry and does not upload data, projects, models, results, or formulas. Its automatic startup and hourly version check requests public latest-release metadata from GitHub. When plugins are loaded, the application also checks the public validated-plugin catalog at startup and hourly. These checks do not upload scientific data. Opening documentation or release notes, downloading an update, reporting an issue, or using Zenodo is otherwise an explicit external action.

Plugins run locally with the user's permissions and can violate these expectations; trust only reviewed plugin code.

### 20.3 Citation

If CurveMole contributes to published work, cite the exact version used. The release DOI is inserted into `CITATION.cff` and the repository README after Zenodo archival. Until archival completes, the versioned GitHub release is the authoritative record:

> Romi, S. (2026). *CurveMole: Modular Scientific Curve Fitting* (Version 0.42.0) [Computer software]. GitHub. https://github.com/SebRoLENS/curvemole/releases/tag/v0.42.0

The repository provides **Cite this repository** from `CITATION.cff`.

### 20.4 Author and contact

Sebastiano Romi
European Laboratory for Non-Linear Spectroscopy (LENS)
University of Florence (UNIFI)
romi@lens.unifi.it

### 20.5 License

CurveMole is released under GPL-3.0-or-later. User data, private projects, results, and unpublished formulas remain under the user's control. Publicly distributed extensions must follow their applicable license obligations.

## Appendix A. Keyboard shortcuts

Shortcuts use the platform’s standard key sequence where applicable.

| Shortcut | Action |
|---|---|
| Ctrl+N | New project |
| Ctrl+O | Open project |
| Ctrl+I | Import data |
| Ctrl+S | Save project |
| Ctrl+Shift+S | Save project as on common desktop mappings |
| Ctrl+E | Export analysis bundle |
| Ctrl+Shift+E | Export one numeric file per selected spectrum |
| Ctrl+Z | Undo |
| Standard Redo shortcut | Redo |
| Ctrl++ | Add component |
| F5 | Open Fit dialog |
| F1 or platform Help shortcut | Quick Start |
| Esc | Cancel graphical peak or manual-point placement |
| Ctrl while dragging a normal peak handle | Change a fixed graphical parameter without unfixing it |
| Double-click during manual-point placement | Finish after the minimum number of points |
| Right-click during manual-point placement | Remove the nearest point |
| Right-drag / left-drag with Mask active | Mask / unmask an interval |
| Up / Down with plot focus | Activate previous / next spectrum |

## Appendix B. Built-in identifiers

| Identifier | Display name | Parameters |
|---|---|---|
| `gaussian` | Gaussian | `area`, `center`, `sigma` |
| `lorentzian` | Lorentzian | `area`, `center`, `gamma` |
| `voigt` | Voigt | `area`, `center`, `sigma`, `gamma` |
| `pseudo_voigt` | Pseudo-Voigt | `area`, `center`, `fwhm`, `eta` |
| `constant` | Constant | `offset` |
| `linear` | Linear | `intercept`, `slope` |
| `polynomial` | Polynomial | `c0` through selected order |
| `cubic_spline` | Cubic spline | `y0` through last node |

## Appendix C. File extensions

| Extension | Purpose |
|---|---|
| `.fitproj` | Complete CurveMole project |
| `.fitmodel` | Reusable model and formula definitions |
| `.curvemole-function.json` | Reusable safe formula in `my_curvemole_functions` |
| `.curvemole-plugin.json` | Manifest for an executable Python plugin |
| `.yml`, `.yaml` | Reproducible workflow |
| Any text-file suffix | Imported data when the contents form a valid numeric table |
| `.whl` | Installable Python package |
| `.AppImage` | Linux desktop package |
| `.exe` | Windows installer or portable executable |
| `.zip` | Windows portable package or plugin download |
| `.dmg` | macOS disk image |

## Appendix D. Preview 0.42.0 limitations

The following boundaries are important when evaluating this release:

- scientific validation toward 1.0.0 is still in progress;
- local optimizers include nonlinear least squares, Nelder-Mead, Powell, and L-BFGS-B; none guarantees the global optimum for an arbitrary nonlinear model;
- `sigma_x` is stored but not used in optimization;
- multi-source relations use the advanced expression editor, with readable parameter references inserted through the source picker and **Add**;
- fit ranges exist in the core model but do not yet have a complete graphical editor;
- the GUI does not yet provide one consolidated table for every fit statistic;
- Windows and macOS packages may be unsigned; Linux release provenance is available through GitHub artifact attestation;
- the interface and manual are currently maintained in English;
- plugin API and other 0.x APIs may change before 1.0.0.

These limitations are stated so that testing can focus on genuine behavior rather than inferred promises. Report unclear workflows and reproducible defects through the issue tracker.

## Appendix E. Building this manual

The Markdown file you are reading is the only hand-edited manual source. From a source checkout with Pandoc, XeLaTeX, and `latexmk` installed, run:

```
python scripts/build_manual.py
```

The command validates version declarations and local links, then creates:

```
docs/CurveMole_User_Manual.tex
docs/CurveMole_User_Manual.pdf
```

GitHub Actions repeats this process for documentation changes and releases. Release assets use versioned filenames and include both LaTeX and PDF editions. Do not edit the generated files directly; changes will be replaced by the next automated build.